UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

CARRERA DE CIENCIAS DE LA COMPUTACIÓN

MBB-GS Moving Gaussians

Representación multimedia mediante Gaussian Splatting

TRABAJO DE INVESTIGACIÓN

Para optar el grado de Bachiller
en Ciencias de la Computación

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

AUTOR(ES)

Jose Leandro Machaca Soloaga

Mauro Ianfranco Bobadilla Castillo

ASESOR(ES)

Eric Biagioli

Lima - Perú
2026

Dedicatoria

Dedicamos esta tesis a nuestras familias, por su apoyo incondicional, su paciencia y por acompañarnos durante cada etapa de nuestra formación profesional.

A Beatriz Quiroz, por su amor, comprensión y apoyo constante a lo largo de este camino. Gracias por acompañarme en los momentos más exigentes y por motivarme a continuar hasta alcanzar esta meta. De manera especial, dedico también este trabajo a la memoria de mi amigo Christofer Condori, quien formó parte de la comunidad UTEC. Me habría gustado que pudiera leer este documento y compartir conmigo la culminación de esta etapa. Su recuerdo permanece presente en este logro.

— Mauro

A Daniela Rodrigo, por su inmensa paciencia en esos momentos en los que el tiempo no me alcanzaba y por apoyarme siempre con su constante «Tú puedes, Yato». Gracias por acompañarme en las amanecidas, por esperarme cada noche para hablar antes de dormir y por tu disposición incondicional para escucharme, incluso cuando no entendías del todo lo que te decía. De manera muy especial, a mi mamá y a mi papá, por acompañarme siempre y respaldar mis decisiones, fueran buenas o malas; por esperarme con un plato de comida y por sus infaltables deseos de buena suerte antes de salir a estudiar o trabajar.

— Leandro

Agradecimientos

Agradecemos a la Universidad de Ingeniería y Tecnología (UTEC) y a nuestros profesores por la formación y los conocimientos brindados durante nuestra etapa universitaria.

A nuestro asesor, Eric Biagoli, por su constante apoyo y por nutrir este proyecto con nuevas ideas. Agradecemos profundamente su paciencia para explicarnos y la total franqueza y claridad con la que siempre guió nuestro trabajo. Su nivel de exigencia, sus oportunas correcciones y su continuo aliento fueron los pilares que hicieron posible el buen resultado de esta investigación.

— Mauro y Leandro

El trabajo computacional del presente proyecto fue apoyado por los recursos del clúster HPC Khipu (https://web.khipu.utec.edu.pe/) de la Universidad de Ingeniería y Tecnología (UTEC), cuya infraestructura fue fundamental para la realización de nuestros experimentos.

Resumen

El video suele representarse como una secuencia de *frames*. Aunque esta forma es útil para almacenarlo y reproducirlo, no describe directamente cómo cambian los elementos visuales a lo largo del tiempo. Esta tesis estudia una forma alternativa de representar video mediante *Gaussian Splatting* 2D temporal.

La propuesta utiliza un conjunto de gaussianas bidimensionales, donde cada gaussiana representa una pequeña región visual de la imagen. A diferencia de una representación por *frames* independientes, los atributos de cada gaussiana, como posición, color, opacidad, escala, rotación y profundidad, cambian en el tiempo mediante polinomios de Chebyshev. De esta manera, el video puede entenderse como una señal visual continua que puede evaluarse en distintos instantes.

El modelo fue implementado mediante un rasterizador diferenciable adaptado y acelerado en GPU. Se realizaron experimentos para analizar la influencia de la base temporal, el número de gaussianas, los grados de los polinomios y las funciones de pérdida. En particular, se observó que una función de pérdida que da mayor importancia a las regiones con movimiento mejora la reconstrucción de las zonas dinámicas del video.

Además, se exploraron aplicaciones derivadas de esta representación, como la interpolación temporal y la reducción del modelo. La interpolación se obtiene al evaluar el modelo en instantes intermedios, sin entrenar una red adicional. Sin embargo, esta línea todavía presenta limitaciones en escenas con movimiento rápido, por lo que se considera una dirección prometedora para trabajo futuro.

Como extensión complementaria, se estudió la representación de audio mediante átomos de Gabor. Estos permiten aproximar la forma de onda usando componentes con amplitud, frecuencia, duración y fase. Los resultados muestran que los átomos de Gabor son más adecuados que las gaussianas puras para representar señales con oscilaciones.

En conjunto, esta tesis muestra que las primitivas explícitas pueden utilizarse como una alternativa para representar señales multimedia, con el video como eje principal del trabajo.

Palabras clave: Gaussian Splatting; representación de video; representación temporal; interpolación temporal; polinomios de Chebyshev; átomos de Gabor; señales multimedia.

Abstract

Video is commonly represented as a sequence of frames. Although this format is useful for storage and playback, it does not directly describe how visual elements change over time. This thesis studies an alternative way to represent video using temporal 2D Gaussian Splatting.

The proposed model uses a set of two-dimensional Gaussians, where each Gaussian represents a small visual region of the image. Instead of storing an independent representation for each frame, the attributes of each Gaussian, such as position, color, opacity, scale, rotation, and depth, vary over time using Chebyshev polynomials. In this way, the video can be understood as a continuous visual signal that can be evaluated at different time instants.

The model was implemented using an adapted differentiable rasterizer accelerated on GPU. Several experiments were conducted to analyze the effect of the temporal basis, the number of Gaussians, the polynomial degrees, and the loss functions. In particular, a loss function that gives more importance to moving regions improved the reconstruction of dynamic parts of the video.

Additionally, this work explores applications derived from the proposed representation, such as temporal interpolation and model reduction. Temporal interpolation is obtained by evaluating the trained model at intermediate time instants, without training an additional interpolation network. However, this line still presents limitations in scenes with fast motion, so it is considered a promising direction for future work.

As a complementary extension, audio representation using Gabor atoms was also studied. These atoms approximate the waveform using components with amplitude, frequency, duration, and phase. The results show that Gabor atoms are more suitable than pure Gaussians for representing signals with oscillations.

Overall, this thesis shows that explicit primitives can be used as an alternative way to represent multimedia signals, with video as the main focus of the work.

Keywords: Gaussian Splatting; video representation; temporal representation; temporal interpolation; Chebyshev polynomials; Gabor atoms; multimedia signals.

TABLA DE CONTENIDO

Pág.

CAPÍTULO Resumen III

CAPÍTULO Abstract v

CAPÍTULO I Introducción 1

1.1 Descripción del problema 1

1.2 Motivación y objetivos 2

1.3 Contribuciones y estructura del trabajo 3

CAPÍTULO II Fundamentos teóricos 5

2.1 Primitivas gaussianas y splatting 5

2.2 Gaussian Splatting 3D 6

2.3 Gaussian Splatting 2D 7

2.4 Polinomios de Chebyshev y parametrización temporal 8

2.5 Átomos de Gabor 10

2.6 Métricas de evaluación 11

CAPÍTULO III Estado del arte 13

3.1 Representación de imágenes mediante Gaussian Splatting 2D 13

3.2 Representación temporal de video con gaussianas 13

3.3 Interpolación temporal de video 14

3.4 Representación de audio mediante primitivas 14

3.5 Posicionamiento de la propuesta 15

CAPÍTULO IV Implementación del modelo de video 16

4.1 Reproducción del Gaussian Splatting 3D 16

4.2 Adaptación del rasterizador diferenciable y aceleración en CUDA 17
4.3 Gaussian Splatting 2D para imágenes 19
4.4 Modelo: atributos de las gaussianas como funciones del tiempo 20

CAPÍTULO V Experimentos de representación de video 24

5.1 Base monomial frente a base de Chebyshev (Experimento 1) 24
5.2 Grados del polinomio y número de gaussianas (Experimento 2) 26
5.3 Función de pérdida: ablación por fases y peso por movimiento (Experimento 3) 30

CAPÍTULO VI Aplicaciones del framework de video 34

6.1 Reconstrucción/interpolación de frames y cámara lenta 34
6.2 Poda de gaussianas y reducción del modelo 36

CAPÍTULO VII Representación de audio con átomos de Gabor 40

7.1 Intento con Chebyshev sobre el espectrograma y sus limitaciones (Experimento 4) 40
7.2 Modelo de átomos de Gabor (Experimento 5) 42
7.3 Ablaciones: pérdida, inicialización y mono/estéreo (Experimento 6) 44
7.3.1 Pérdida de entrenamiento 44
7.3.2 Inicialización 45
7.3.3 Mono y estéreo 45
7.3.4 Ejemplo comparativo en un fragmento musical 46

CAPÍTULO VIII Representación multimedia integrada 51

8.1 Integración del pipeline: Chebyshev y átomos de Gabor 51
8.2 Validación del sistema audiovisual 52

CAPÍTULO IX Conclusiones y trabajo futuro 59

9.1 Conclusiones 59
9.2 Limitaciones y trabajo futuro 61

CAPÍTULO A Repositorio de código y reproducibilidad 67

1.1 Disponibilidad del código fuente 67
1.2 Contenido general del repositorio 67

1.3 Versión del código asociada con la tesis 68
1.4 Requisitos generales de ejecución 69
1.5 Datos, resultados y material complementario 69

ÍNDICE DE TABLAS

Tabla 5.1 Comparación entre base monomial y base de Chebyshev. 25
Tabla 5.2 Resultados del Experimento 2 al variar el número de gaussianas y los grados temporales. 27
Tabla 5.3 Resultados de la ablación de funciones de pérdida. 32
Tabla 5.4 Resultados al variar el exponente utilizado para combinar el error entre frames. 33

Tabla 6.1 Métricas de reconstrucción del modelo usado para interpolación temporal. 35
Tabla 6.2 Métricas de la reconstrucción completa antes de la reducción del modelo. 37
Tabla 6.3 Comparación de tamaño entre el modelo completo y la versión reducida cuantizada. 38

Tabla 7.1 Comparación de métricas entre una configuración reducida y una configuración final de átomos de Gabor sobre un fragmento estéreo de 9 segundos. 47

Tabla 8.1 Métricas de reconstrucción de video en el sistema audiovisual. 53
Tabla 8.2 Métricas de reconstrucción de audio en el sistema audiovisual. 54

ÍNDICE DE FIGURAS

Figura 5.1 Comparación visual entre la reconstrucción con base de Chebyshev (izquierda) y base monomial (derecha). 26
Figura 5.2 Comparación visual del efecto de aumentar el número de gaussianas en la reconstrucción del video. . . 28

Figura 7.1 Comparación de la forma de onda original y la reconstrucción obtenida con la configuración reducida de Gabor. 48
Figura 7.2 Espectrograma del experimento con configuración reducida de Gabor. 49
Figura 7.3 Comparación de la forma de onda original y la reconstrucción obtenida con la configuración final de Gabor. 49
Figura 7.4 Espectrograma del experimento con configuración final de Gabor. 50

Figura 8.1 Comparación entre la señal de audio original y la señal reconstruida mediante átomos de Gabor. 55
Figura 8.2 Comparación entre el espectrograma del audio original y el espectrograma del audio reconstruido. . . . 55
Figura 8.3 Comparación visual del *frame* 23 entre el video original, la reconstrucción completa y las reconstrucciones usando subconjuntos de 80k y 20k gaussianas importantes. 56

Figura 8.4 Comparación visual del *frame* 162 entre el video original, la reconstrucción completa y las reconstrucciones usando subconjuntos de 80k y 20k gaussianas importantes. 57

CAPÍTULO I

Introducción

1.1 Descripción del problema

Un video suele representarse como una secuencia de imágenes o *frames*. Esta forma es útil para almacenarlo y reproducirlo, pero no describe de manera directa cómo cambian los elementos visuales a lo largo del tiempo. Por ejemplo, si una persona se desplaza o un objeto cambia de tamaño, el video muestra ese cambio en varios *frames*, pero no expresa explícitamente la función que describe dicho movimiento.

Esta limitación se vuelve relevante cuando se busca reconstruir instantes intermedios, estudiar el movimiento o modificar la evolución de los elementos visuales. En estos casos, analizar cada *frame* de manera aislada no es suficiente, ya que también es necesario representar la relación temporal entre ellos.

En esta tesis se propone representar el video mediante un conjunto de primitivas gaussianas. De manera sencilla, cada gaussiana puede entenderse como una pequeña región de la imagen que tiene una posición, un tamaño, un color, una orientación y una transparencia. La combinación de muchas de estas regiones permite reconstruir el contenido visual.

El problema principal consiste en definir cómo deben cambiar estas gaussianas con el tiempo para que un mismo modelo pueda representar una parte del video. En lugar de almacenar una representación independiente para cada *frame*, se busca que los atributos de las gaussianas sean funciones temporales que describan su evolución durante la secuencia.

De esta manera, la investigación plantea una forma distinta de entender el video: no solo como una colección de imágenes, sino como una señal visual continua cuyos componentes cambian en función del tiempo. Esta representación puede

servir como base para distintas aplicaciones relacionadas con la reconstrucción, la interpolación y el análisis temporal del video.

1.2 Motivación y objetivos

La principal motivación de esta investigación surge de la necesidad de explorar una forma diferente de representar el video. Aunque una secuencia de *frames* permite observar el contenido en distintos instantes, esta representación no describe directamente cómo cambian sus elementos a lo largo del tiempo.

Representar estos cambios mediante funciones temporales permitiría describir de forma continua el movimiento y la evolución de la apariencia de los objetos. De esta manera, el modelo podría ser evaluado no solo en los instantes originales del video, sino también en momentos intermedios.

Las primitivas gaussianas ofrecen una base adecuada para estudiar esta idea, debido a que cada una representa una parte visible de la imagen mediante atributos sencillos, como su posición, color, tamaño, orientación y transparencia. Además, estos atributos pueden ajustarse automáticamente a partir del video mediante métodos de optimización.

Por ello, el objetivo general de esta tesis es desarrollar y evaluar una nueva forma de representar videos mediante gaussianas bidimensionales cuyos atributos sean funciones del tiempo.

Para alcanzar este objetivo general, se plantean los siguientes objetivos específicos:

- Diseñar una representación temporal en la que la posición, el color, la opacidad, la escala y la orientación de cada gaussiana puedan cambiar a lo largo del video.
- Implementar un sistema de renderizado acelerado en GPU que permita reconstruir los diferentes instantes del video a partir de las gaussianas y sus funciones temporales.

- Evaluar distintas formas de parametrizar la evolución temporal, incluyendo el uso de polinomios de Chebyshev.
- Analizar la influencia del número de gaussianas, los grados de las funciones temporales y las funciones de pérdida sobre la calidad de la representación.
- Evaluar la capacidad del modelo para reconstruir los *frames* originales y representar instantes intermedios de la secuencia.
- Explorar la aplicación del mismo principio general de representación mediante primitivas en una segunda modalidad multimedia, utilizando átomos de Gabor para representar señales de audio.

1.3 Contribuciones y estructura del trabajo

El trabajo concreta la propuesta anterior mediante un modelo de gaussianas 2D con parámetros temporales. Para describir la evolución de estos parámetros se utilizaron polinomios de Chebyshev, los cuales fueron incorporados como una base matemática para modelar los cambios de posición, color, opacidad, escala, orientación y profundidad.

Asimismo, se adaptó el proceso de rasterización de *Gaussian Splatting* a las necesidades del modelo bidimensional y temporal. Esta adaptación toma como referencia la lógica del *Gaussian Splatting* original, pero fue ajustada para trabajar directamente con gaussianas 2D y atributos dependientes del tiempo. Además, se implementó una versión acelerada en CUDA para hacer viable el entrenamiento en GPU.

También se realizaron experimentos para estudiar la influencia de la base polinómica, los grados temporales, el número de gaussianas y las funciones de pérdida. Estas evaluaciones permitieron analizar qué configuraciones favorecen la reconstrucción del video, especialmente en regiones con movimiento.

Como extensión complementaria, se aplicó el mismo principio general a señales de audio mediante átomos de Gabor. En este caso, los parámetros de los átomos se optimizan para aproximar directamente la forma de onda, incluyendo su amplitud, frecuencia y fase. Para esta modalidad también se desarrolló una implementación CUDA que permite acelerar el cálculo de la señal y de sus gradientes.

Como parte de los resultados de esta investigación, el código fuente desarrollado para MBB-GS se encuentra disponible públicamente en el repositorio oficial del proyecto. Este incluye la implementación de los modelos de representación de video y audio, los módulos de aceleración en CUDA, así como los scripts y archivos de configuración utilizados durante el desarrollo experimental. El repositorio puede consultarse en:

https://github.com/JLeandroJM/MBB-GS

El resto del documento se organiza de la siguiente manera. El Capítulo II presenta los fundamentos teóricos necesarios para comprender la propuesta. El Capítulo III revisa los trabajos relacionados y posiciona esta tesis dentro del estado del arte. El Capítulo IV describe la implementación del modelo de video, incluyendo la adaptación del rasterizador diferenciable y su aceleración en CUDA.

El Capítulo V presenta los experimentos realizados sobre la representación de video. El Capítulo VI estudia aplicaciones derivadas del modelo, como la interpolación temporal y la reducción del modelo. El Capítulo VII desarrolla la representación de audio mediante átomos de Gabor. El Capítulo VIII presenta la integración de las modalidades de audio y video. Finalmente, el Capítulo IX reúne las conclusiones, las limitaciones y las posibles líneas de trabajo futuro.

CAPÍTULO II

Fundamentos teóricos

Este capítulo presenta los conceptos necesarios para comprender la representación propuesta en esta tesis. Primero se introducen las primitivas gaussianas y la idea general de *splatting*. Luego se revisa el *Gaussian Splatting* tridimensional y su formulación bidimensional. Finalmente, se presentan los polinomios de Chebyshev, utilizados para describir cambios en el tiempo, y los átomos de Gabor, empleados como base para la representación de señales de audio.

2.1 Primitivas gaussianas y splatting

Una gaussiana es una función suave cuyo valor es mayor cerca de su centro y disminuye gradualmente a medida que aumenta la distancia. En el contexto de imágenes y video, una gaussiana puede entenderse como una pequeña región visual que contribuye con color y transparencia sobre una zona de la imagen.

De manera general, una gaussiana centrada en una posición $\boldsymbol{\mu}$ se puede expresar como

$$G(\mathbf{x}) = \exp\left(-\frac{1}{2}(\mathbf{x}-\boldsymbol{\mu})^\top \boldsymbol{\Sigma}^{-1}(\mathbf{x}-\boldsymbol{\mu})\right), \tag{II.1}$$

donde $\mathbf{x}$ es el punto en el que se evalúa la función, $\boldsymbol{\mu}$ representa el centro de la gaussiana y $\boldsymbol{\Sigma}$ es la matriz de covarianza. Esta matriz controla la forma, el tamaño y la orientación de la gaussiana.

La idea de *splatting* consiste en proyectar o colocar estas primitivas sobre una imagen y combinar sus contribuciones para formar el resultado final. En lugar de asignar un único valor a un píxel, cada primitiva puede influir sobre una

región cercana. Por ello, el resultado se obtiene como la superposición de muchas contribuciones suaves.

Esta representación es útil porque sus parámetros pueden ser modificados durante un proceso de optimización. Si la imagen generada no coincide con una referencia, es posible calcular cómo deben cambiar las posiciones, colores, opacidades o escalas de las gaussianas para reducir el error.

2.2 Gaussian Splatting 3D

El *3D Gaussian Splatting* representa una escena tridimensional mediante un conjunto de gaussianas ubicadas en el espacio [1]. Cada gaussiana posee una posición, una escala, una orientación, un color y una opacidad. De forma intuitiva, puede imaginarse como una pequeña nube semitransparente que ocupa una región del espacio.

La posición central de una gaussiana tridimensional se representa mediante

$$\boldsymbol{\mu} = \begin{bmatrix} \mu_x & \mu_y & \mu_z \end{bmatrix}^\top . \tag{II.2}$$

Su forma y orientación se describen mediante una matriz de covarianza $\boldsymbol{\Sigma}$. Esta matriz indica en qué direcciones se extiende la gaussiana y qué tan grande es en cada una de ellas.

Para generar una imagen, las gaussianas 3D se proyectan sobre el plano de una cámara. Después de la proyección, cada gaussiana ocupa una región elíptica en la imagen. Las gaussianas que influyen sobre un mismo píxel se combinan según su opacidad y profundidad.

La composición de colores se realiza mediante *alpha blending*. De manera simplificada, el color final de un píxel puede escribirse como

$$C = \sum_{i=1}^{N} c_i \alpha_i \prod_{j=1}^{i-1} (1 - \alpha_j), \tag{II.3}$$

donde c_i es el color aportado por la gaussiana i y α_i representa su opacidad efectiva sobre el píxel. El producto indica cuánto de la contribución posterior sigue siendo visible después de considerar las gaussianas anteriores.

Una propiedad importante de este proceso es que es diferenciable. Esto significa que se puede calcular cómo cambia la imagen final cuando se modifica alguno de los parámetros de las gaussianas. Gracias a ello, los parámetros pueden ajustarse automáticamente mediante descenso de gradiente.

Debido al gran número de gaussianas y píxeles, el renderizado se organiza en pequeñas regiones de la imagen llamadas *tiles*. Cada *tile* solo procesa las gaussianas que pueden influir en esa zona, lo que reduce el número de cálculos y permite aprovechar el paralelismo de la GPU.

2.3 Gaussian Splatting 2D

Cuando el objetivo es representar una imagen directamente, no siempre es necesario trabajar con una escena tridimensional. Trabajos como *GaussianImage* muestran que una imagen puede representarse mediante gaussianas 2D optimizadas directamente en el plano de la imagen [2]. En una formulación bidimensional, las gaussianas se colocan sobre el plano de la imagen, sin usar cámaras ni proyecciones 3D.

La posición de una gaussiana 2D se define como

$$\boldsymbol{\mu}_i = \begin{bmatrix} \mu_{x,i} & \mu_{y,i} \end{bmatrix}^\top, \tag{II.4}$$

donde $\mu_{x,i}$ y $\mu_{y,i}$ indican su ubicación dentro de la imagen.

La influencia de esta gaussiana sobre un píxel ubicado en la posición $\mathbf{p}$ se calcula mediante

$$G_i(\mathbf{p}) = \exp\left(-\frac{1}{2}(\mathbf{p}-\boldsymbol{\mu}_i)^\top \boldsymbol{\Sigma}_i^{-1}(\mathbf{p}-\boldsymbol{\mu}_i)\right). \tag{II.5}$$

El valor de $G_i(\mathbf{p})$ es alto cuando el píxel está cerca del centro de la gaussiana y disminuye suavemente cuando se aleja. La matriz $\boldsymbol{\Sigma}_i$ controla si la región es más circular, alargada o inclinada.

En una imagen, cada gaussiana puede aportar color y opacidad. Al combinar muchas gaussianas, se obtiene una reconstrucción visual. Si además se asigna un valor de profundidad, es posible definir un orden de composición similar al utilizado en *Gaussian Splatting* 3D.

La diferencia principal frente al caso tridimensional es que la posición de las gaussianas ya se encuentra directamente en coordenadas de imagen. Por ello, no se requiere una etapa de proyección desde el espacio 3D. Esta simplificación resulta adecuada cuando el objetivo no es reconstruir una escena desde múltiples vistas, sino representar contenido visual en el plano de la imagen.

2.4 Polinomios de Chebyshev y parametrización temporal

Para representar un video, no basta con describir una imagen en un único instante. Es necesario modelar cómo cambian sus elementos a lo largo del tiempo. Una forma de hacerlo es expresar los atributos de cada primitiva como funciones temporales.

Por ejemplo, la posición horizontal de una gaussiana puede representarse como una función $\mu_x(t)$, mientras que su opacidad, color o escala pueden tener sus propias funciones del tiempo. El modelo aprende los parámetros de estas funciones a partir del video.

Una forma simple de construir una función temporal es usar una base monomial:

$$p(t) = a_0 + a_1 t + a_2 t^2 + \cdots + a_K t^K, \tag{II.6}$$

donde $p(t)$ representa un atributo cualquiera y $a_0, a_1, \ldots, a_K$ son coeficientes.

Aunque esta forma es fácil de entender, las potencias altas de t pueden generar problemas numéricos, especialmente cuando se utilizan grados elevados. Algunos términos pueden dominar el resultado, mientras que otros tienen muy poca influencia, lo que dificulta la optimización.

Los polinomios de Chebyshev ofrecen una base alternativa. Los polinomios de Chebyshev de primer tipo se definen en el intervalo $[-1, 1]$ mediante la recurrencia

$$T_0(x) = 1, \qquad T_1(x) = x, \qquad T_{k+1}(x) = 2xT_k(x) - T_{k-1}(x). \tag{II.7}$$

Los primeros polinomios son

$$T_0(x) = 1, \qquad T_1(x) = x, \qquad T_2(x) = 2x^2 - 1, \qquad T_3(x) = 4x^3 - 3x. \tag{II.8}$$

Antes de utilizar esta base, el tiempo se normaliza al intervalo $[-1, 1]$. Si t representa el índice de un *frame* y el video contiene F *frames*, se puede utilizar

$$\tau(t) = 2\frac{t}{F-1} - 1. \tag{II.9}$$

De esta manera, el primer *frame* se ubica en -1 y el último en 1.

Un atributo temporal puede expresarse entonces como

$$p(t) = \sum_{k=0}^{K} a_k T_k \left(\tau(t)\right), \tag{II.10}$$

donde K es el grado de la función y los coeficientes a_k son los parámetros que se aprenden durante la optimización.

Esta formulación permite describir cambios temporales de diferente complejidad. Por ejemplo, un atributo de posición puede requerir un grado mayor que un atributo de profundidad o rotación. Por ello, en un modelo temporal es posible asignar distintos grados a distintos grupos de atributos.

2.5 Átomos de Gabor

Una señal de audio puede entenderse como una secuencia de valores que cambia en el tiempo. A diferencia de una imagen, el audio contiene oscilaciones rápidas asociadas con distintas frecuencias. Por esta razón, una gaussiana pura no suele ser suficiente para representar eficientemente una señal sonora, ya que no posee una componente oscilatoria.

Un átomo de Gabor combina una envolvente gaussiana con una función cosenoidal [3]. Su expresión es

$$\psi_i(t) = A_i \exp\left(-\frac{(t-\mu_i)^2}{2\sigma_i^2}\right) \cos\left(2\pi f_i(t-\mu_i) + \varphi_i\right), \tag{II.11}$$

donde μ_i indica la posición temporal del átomo, σ_i controla su duración, A_i representa su amplitud, f_i determina su frecuencia y φ_i corresponde a su fase.

La parte gaussiana permite localizar el átomo en una región del tiempo. La parte cosenoidal permite representar oscilaciones dentro de esa región. Por ello, un átomo de Gabor puede describir componentes sonoras localizadas tanto en tiempo como en frecuencia.

Una señal puede aproximarse mediante la suma de varios átomos:

$$\hat{x}(t) = \sum_{i=1}^{N} \psi_i(t), \tag{II.12}$$

donde $\hat{x}(t)$ es la señal reconstruida y N es el número de átomos utilizados.

Una característica importante de esta formulación es que la fase forma parte de los parámetros del átomo. Por ello, la señal puede reconstruirse directamente en el dominio temporal, sin requerir una etapa separada para estimar la fase.

Los átomos de Gabor también se relacionan con métodos de descomposición como *matching pursuit* [4], donde una señal se aproxima seleccionando átomos de un diccionario. En esta tesis, esta idea sirve como antecedente conceptual para representar audio mediante una suma de primitivas localizadas.

2.6 Métricas de evaluación

Para evaluar la calidad de las reconstrucciones obtenidas se utilizaron métricas diferentes según el tipo de señal. En el caso de imágenes y video se emplearon principalmente PSNR y SSIM, mientras que para audio se utilizó SNR.

El *Peak Signal-to-Noise Ratio* (PSNR) mide la relación entre el valor máximo posible de una imagen y el error cuadrático medio entre la imagen original y la imagen reconstruida. Se expresa en decibeles (dB). Un valor mayor de PSNR indica que la reconstrucción presenta menos error respecto al original. Por esta razón, esta métrica se utiliza frecuentemente para comparar la calidad de reconstrucción de imágenes y *frames* de video.

El *Structural Similarity Index Measure* (SSIM) evalúa la similitud estructural entre dos imágenes. A diferencia del PSNR, que se basa directamente en el error píxel a píxel, SSIM considera información relacionada con luminancia, contraste y estructura. Sus valores suelen estar entre 0 y 1, donde un valor más cercano a 1 indica mayor similitud con la imagen original.

En el caso del audio, se utilizó el *Signal-to-Noise Ratio* (SNR), que mide la relación entre la energía de la señal original y la energía del error de reconstrucción. También se expresa en decibeles (dB). Un valor mayor de SNR indica que la señal reconstruida conserva mejor la información del audio original y que el error introducido por el modelo es menor.

Además del SNR, para audio se utilizó SI-SDR, una métrica más robusta frente a cambios de escala en la señal reconstruida. También se emplearon métricas espectrales como LSD, Mel-L1 y MR-STFT, ya que permiten evaluar diferencias en la distribución de frecuencias y no solo el error muestra a muestra en la forma de onda.

El LSD mide la distancia entre espectros en escala logarítmica, por lo que ayuda a identificar diferencias relacionadas con el timbre. Mel-L1 compara representaciones en escala Mel, más cercana a la percepción auditiva. Por su parte, MR-STFT calcula errores usando transformadas de Fourier de corto tiempo con distintas resoluciones, lo que permite evaluar componentes temporales y frecuenciales de la señal.

En el caso de audio estéreo, también se reportó PSNR estéreo, calculado sobre la señal reconstruida en ambos canales. Estas métricas se usaron de manera complementaria, ya que una sola métrica no describe completamente la calidad perceptual del audio.

Estas métricas permiten comparar de forma cuantitativa distintas configuraciones del modelo. Sin embargo, también es importante complementar su interpretación con una revisión visual o auditiva de los resultados, ya que una métrica alta no siempre garantiza una percepción perfecta de la reconstrucción.

CAPÍTULO III

Estado del arte

Este capítulo revisa los trabajos más relacionados con la representación de imágenes, video y audio mediante primitivas optimizables.

3.1 Representación de imágenes mediante Gaussian Splatting 2D

Un antecedente directo es *GaussianImage*, que muestra que una imagen puede representarse mediante gaussianas 2D optimizadas por gradiente [2]. Este trabajo es importante porque traslada el principio de *Gaussian Splatting* al plano de la imagen, sin necesidad de reconstruir una escena tridimensional.

Sin embargo, *GaussianImage* trabaja con imágenes estáticas. Por ello, no modela la evolución temporal de los atributos ni permite representar un video como una señal continua. En esta tesis, esta idea se toma como punto de partida para extender la representación hacia secuencias de video.

3.2 Representación temporal de video con gaussianas

Trabajos recientes como *GSVC*, *GaussianVideo*, *D2GV* y *GSVR* exploran el uso de gaussianas para representar video [5–8]. Estas propuestas demuestran que las gaussianas pueden utilizarse más allá de imágenes estáticas y que es posible incorporar información temporal en la representación.

No obstante, varios de estos enfoques siguen organizándose alrededor de *frames*, grupos de imágenes, deformaciones aprendidas, campos de movimiento o esquemas de compresión. En contraste, esta tesis estudia una formulación directa: una población de gaussianas 2D cuyos atributos son funciones del tiempo. Así, el video se interpreta como una función evaluable en distintos instantes.

3.3 Interpolación temporal de video

La interpolación temporal suele abordarse mediante métodos de estimación de movimiento o redes entrenadas para generar *frames* intermedios. Métodos como *Super SloMo* y *RIFE* son ejemplos representativos de esta línea [9, 10].

También existen trabajos recientes, como *GS-STVSR*, que relacionan *Gaussian Splatting* 2D con super-resolución espacial y temporal [11]. Esta línea es cercana a la presente tesis porque trabaja con representaciones continuas y aumento de FPS.

La diferencia principal es que, en esta tesis, la interpolación aparece como consecuencia del modelo temporal aprendido. No se entrena una red adicional de interpolación ni se aplica un método externo de flujo óptico; simplemente se evalúa la representación del video en tiempos intermedios.

3.4 Representación de audio mediante primitivas

En audio existe una tradición previa de representar señales como suma de átomos paramétricos. Un ejemplo clásico es *matching pursuit*, donde una señal se aproxima seleccionando progresivamente átomos de un diccionario [4]. Dentro de este tipo de representaciones, los átomos de Gabor son especialmente adecuados porque combinan localización temporal con una componente oscilatoria [3].

Cuando una representación trabaja únicamente sobre la magnitud del espectrograma, la reconstrucción de audio requiere recuperar o estimar la fase. Métodos como Griffin-Lim permiten estimar la fase a partir de la magnitud, pero pueden introducir artefactos y no garantizan una reconstrucción perfecta de la señal temporal [12].

Por otro lado, trabajos modernos como *DDSP* muestran que los modelos de procesamiento de señales pueden integrarse con optimización diferenciable [13]. Esta idea se relaciona con la extensión de audio de esta tesis, donde la forma de onda se aproxima mediante átomos de Gabor optimizados por gradiente.

A diferencia de *DDSP*, esta tesis no busca diseñar un sintetizador neuronal de audio, sino representar una señal dada mediante primitivas explícitas optimizables. Sin embargo, ambos enfoques comparten la idea de combinar procesamiento de señales con optimización diferenciable.

3.5 Posicionamiento de la propuesta

La propuesta de esta tesis se diferencia de los trabajos anteriores en que el video se modela como una función temporal continua. En lugar de representar cada *frame* por separado, se mantiene una población de gaussianas 2D cuyos atributos cambian mediante funciones del tiempo.

Esta formulación permite reconstruir los *frames* originales y evaluar el modelo en instantes intermedios. Por ello, la interpolación temporal no se trata como una tarea externa, sino como una aplicación natural de la representación aprendida.

En resumen, el aporte principal se ubica entre las representaciones explícitas basadas en gaussianas, los modelos temporales de video y las representaciones de señales mediante primitivas optimizables.

CAPÍTULO IV

Implementación: del Gaussian Splatting 3D al modelo de video

Este capítulo describe las etapas de implementación que permitieron llegar al modelo de video propuesto. El desarrollo comenzó con la reproducción de los componentes principales de *Gaussian Splatting* 3D, con el objetivo de comprender su funcionamiento desde una perspectiva práctica.

Posteriormente, se adaptó el proceso de rasterización para trabajar directamente sobre imágenes bidimensionales. Una vez validada esta representación en imágenes estáticas, se incorporaron funciones temporales a los atributos de las gaussianas para representar una secuencia de video.

4.1 Reproducción del Gaussian Splatting 3D

La primera etapa del desarrollo consistió en estudiar la implementación de *Gaussian Splatting* 3D y reproducir sus componentes principales. Esta etapa tuvo como propósito comprender cómo se relacionan el modelo de gaussianas, el rasterizador y el proceso de optimización durante la reconstrucción de una escena.

En particular, se revisó el flujo que comienza con la definición de los parámetros de las gaussianas, continúa con la generación de una imagen mediante el rasterizador y termina con el cálculo de una función de pérdida. A partir de este error, los gradientes permiten actualizar los parámetros del modelo durante el entrenamiento.

De manera general, este proceso puede expresarse como

$$\hat{I} = R(\mathcal{G}, \mathcal{C}), \tag{IV.1}$$

donde $\mathcal{G}$ representa el conjunto de gaussianas, $\mathcal{C}$ contiene la información de la cámara, R es el proceso de rasterización y $\hat{I}$ es la imagen generada.

Luego, la imagen reconstruida se compara con una imagen de referencia:

$$\mathcal{L} = \mathcal{D}(\hat{I}, I), \tag{IV.2}$$

donde I es la imagen objetivo y $\mathcal{D}$ representa la función utilizada para medir la diferencia entre ambas imágenes.

La reproducción permitió verificar que la imagen generada dependiera de los parámetros de las gaussianas y que estos pudieran actualizarse mediante gradientes. También permitió comprender la relación entre la rasterización, la composición de las gaussianas y el proceso de entrenamiento.

A partir de esta experiencia se identificaron los componentes que podían conservarse para el desarrollo posterior, como la optimización por gradiente y la composición de las contribuciones de las gaussianas. En cambio, la proyección tridimensional y el uso de múltiples cámaras no eran necesarios para representar directamente una imagen o un video en dos dimensiones.

Esta etapa sirvió, por tanto, como base práctica para desarrollar el rasterizador bidimensional presentado en la siguiente sección.

4.2 Adaptación del rasterizador diferenciable y aceleración en CUDA

Para generar una imagen a partir de las gaussianas se implementó un rasterizador diferenciable. Este módulo recibe los parámetros de cada gaussiana y produce el *frame* reconstruido. La implementación toma como referencia la idea general del rasterizador de *Gaussian Splatting* 3D [1], pero fue adaptada para trabajar directamente con gaussianas 2D.

Cada gaussiana del modelo se describe mediante los siguientes atributos:

- posición: indica dónde se ubica la gaussiana en la imagen;
- escala: controla su tamaño en dos direcciones;
- rotación: define la orientación de la gaussiana;
- color: indica el color que aporta a la imagen;
- opacidad: controla qué tan visible es;
- profundidad: define el orden en el que se combina con otras gaussianas.

Durante el renderizado, una gaussiana no afecta toda la imagen, sino solo una región cercana a su centro. Para controlar esa región se utiliza un parámetro k_σ, que indica cuántas veces se extiende la gaussiana respecto a su escala. Por ejemplo, un valor mayor de k_σ permite que la gaussiana influya sobre una zona más amplia, mientras que un valor menor reduce su área de influencia.

Evaluar todas las gaussianas sobre todos los píxeles sería demasiado costoso. Por esta razón, la imagen se divide en pequeñas regiones llamadas *tiles*. En una etapa previa al renderizado, se calcula qué *tiles* toca cada gaussiana según su posición, escala, rotación y k_σ. Luego se guarda una lista con los identificadores de las gaussianas que pertenecen a cada *tile*.

Para organizar esta información se utilizan dos estructuras principales. La primera contiene los identificadores de las gaussianas que deben procesarse. La segunda indica el rango de posiciones que corresponde a cada *tile*. Gracias a esto, durante el renderizado cada *tile* puede acceder solo a las gaussianas que realmente pueden afectar sus píxeles.

Además, las gaussianas se ordenan por *tile* y por profundidad. Este ordenamiento permite que, dentro de cada región de la imagen, las gaussianas se combinen en un orden definido. La combinación se realiza mediante opacidad, de forma similar al *alpha blending* usado en *Gaussian Splatting*.

El rasterizador implementa dos etapas principales. En el *forward*, se evalúan las gaussianas y se genera el *frame* reconstruido. En el *backward*, se calculan los

gradientes que indican cómo deben cambiar sus parámetros para reducir el error frente al *frame* objetivo. Estos gradientes se propagan luego hacia las funciones temporales que producen los atributos de cada gaussiana.

La implementación fue desarrollada como una extensión CUDA conectada con PyTorch. Los parámetros principales de las gaussianas se manejan como tensores en punto flotante de 32 bits, mientras que los identificadores de gaussianas y los rangos por *tile* se manejan como enteros. Esta separación permite mantener los valores continuos necesarios para la optimización y, al mismo tiempo, organizar de forma eficiente qué gaussianas procesa cada región de la imagen.

El uso de CUDA fue importante porque el entrenamiento requiere renderizar muchos *frames* y actualizar miles de gaussianas. Al procesar píxeles, *tiles* y gaussianas en paralelo dentro de la GPU, el tiempo de entrenamiento se reduce y se vuelve posible realizar los experimentos de video.

4.3 Gaussian Splatting 2D para imágenes

Una vez comprendido el flujo general de *Gaussian Splatting* 3D y adaptado el rasterizador, la siguiente etapa consistió en representar una imagen estática mediante gaussianas ubicadas directamente sobre el plano bidimensional.

A diferencia del caso tridimensional, en esta implementación no fue necesario utilizar cámaras ni proyectar puntos desde el espacio 3D. La posición de cada gaussiana se definió directamente mediante sus coordenadas dentro de la imagen. También se mantuvieron atributos como la escala, la orientación, el color, la opacidad y la profundidad utilizada para establecer el orden de composición.

Al inicio del entrenamiento se crea un conjunto de gaussianas con parámetros iniciales. A partir de ellas, el rasterizador genera una imagen reconstruida, la cual se compara con una imagen de referencia mediante una función de pérdida. El error obtenido se propaga hacia los parámetros de las gaussianas y estos se actualizan de forma iterativa.

El proceso general puede resumirse de la siguiente manera:

1. Se inicializan las posiciones y los demás atributos de las gaussianas.
2. El rasterizador genera una imagen a partir de su estado actual.
3. La imagen generada se compara con la imagen de referencia.
4. Se calculan los gradientes de la función de pérdida.
5. El optimizador actualiza los parámetros de las gaussianas.

Durante las primeras iteraciones, la reconstrucción presenta formas y colores poco definidos. Sin embargo, a medida que avanza el entrenamiento, las gaussianas modifican su posición, tamaño, orientación, color y opacidad para aproximarse al contenido de la imagen objetivo.

Esta etapa permitió comprobar que el rasterizador bidimensional funcionaba correctamente y que los gradientes llegaban a todos los atributos que debían ser optimizados. También permitió validar el ordenamiento por profundidad y la composición de colores antes de incorporar una dimensión temporal.

La representación de imágenes estáticas fue, por tanto, una etapa intermedia del desarrollo. Una vez que un conjunto de gaussianas pudo aproximar una imagen, el siguiente paso consistió en reemplazar sus atributos constantes por funciones del tiempo. De esta manera, las mismas gaussianas podrían modificar sus propiedades y representar distintos *frames* de una secuencia.

4.4 Modelo: atributos de las gaussianas como funciones del tiempo

Después de validar la representación de una imagen estática, el siguiente paso consistió en incorporar el tiempo al modelo. En la versión anterior, cada gaussiana tenía valores fijos de posición, color, opacidad, escala, orientación y profundidad. Esta configuración permitía representar una sola imagen, pero no los cambios presentes en una secuencia de video.

Para representar distintos instantes, se mantuvo un mismo conjunto de gaussianas y se reemplazaron sus atributos constantes por funciones del tiempo. De esta manera, una gaussiana puede desplazarse, cambiar de tamaño, modificar su color o variar su opacidad durante el video.

Para una gaussiana i y uno de sus atributos p, la parametrización temporal se define como

$$p_i(t) = a_{i,0}^{(p)} + \sum_{k=1}^{K_p} a_{i,k}^{(p)} T_k\left(\tau(t)\right), \tag{IV.3}$$

donde T_k es el polinomio de Chebyshev de grado k, K_p es el grado asignado al atributo y $\tau(t)$ representa el tiempo normalizado al intervalo $[-1, 1]$. Los coeficientes $a_{i,k}^{(p)}$ son los valores que el modelo aprende durante el entrenamiento.

El coeficiente $a_{i,0}^{(p)}$ representa el valor base del atributo, mientras que los coeficientes restantes describen cómo cambia a lo largo del tiempo. Por ejemplo, en el caso de la posición horizontal, la expresión puede escribirse como

$$\mu_{x,i}(t) = a_{i,0}^{(\mu_x)} + \sum_{k=1}^{K_\mu} a_{i,k}^{(\mu_x)} T_k\left(\tau(t)\right). \tag{IV.4}$$

El mismo procedimiento se aplica a los demás atributos. Para cada gaussiana se calculan, en el instante t,

$$\mathcal{G}_i(t) = \{\boldsymbol{\mu}_i(t), \alpha_i(t), \mathbf{c}_i(t), \mathbf{s}_i(t), \theta_i(t), d_i(t)\}, \tag{IV.5}$$

donde:

- $\boldsymbol{\mu}_i(t)$ representa la posición bidimensional;
- $\alpha_i(t)$ corresponde a la opacidad;
- $\mathbf{c}_i(t)$ contiene los componentes de color;

- $\mathbf{s}_i(t)$ representa la escala en ambos ejes;
- $\theta_i(t)$ indica el ángulo de rotación;
- $d_i(t)$ corresponde a la profundidad utilizada para ordenar las gaussianas.

No todos los atributos necesitan cambiar con la misma complejidad. El movimiento de una gaussiana puede requerir una función de mayor grado, mientras que su rotación o profundidad pueden describirse con una función más sencilla. Por esta razón, el modelo permite asignar un grado diferente a cada grupo de atributos.

Para generar un *frame*, primero se selecciona el instante temporal correspondiente y se normaliza su valor. Luego se evalúan los polinomios de Chebyshev para obtener los atributos actuales de todas las gaussianas. Finalmente, estos valores se envían al rasterizador, que genera la imagen reconstruida.

Este proceso puede resumirse como

$$t \longrightarrow \tau(t) \longrightarrow \mathcal{G}(t) \longrightarrow \hat{I}(t), \tag{IV.6}$$

donde $\mathcal{G}(t)$ representa el conjunto de gaussianas evaluadas en el instante t y $\hat{I}(t)$ es el *frame* generado.

Durante el entrenamiento, el *frame* reconstruido se compara con el *frame* original del mismo instante. El error se propaga primero hacia los atributos evaluados y luego hacia los coeficientes de sus funciones temporales. Así, el modelo aprende las funciones que describen la evolución de cada gaussiana durante la secuencia.

La principal característica de esta representación es que no se almacena un conjunto independiente de gaussianas para cada *frame*. En su lugar, las mismas gaussianas se mantienen durante todo el video y son sus funciones temporales las que determinan su estado en cada instante. Esto permite entender el video como una representación continua, que puede evaluarse tanto en los instantes originales como en posiciones intermedias del tiempo.

Con esta implementación, el modelo puede entrenarse directamente sobre una secuencia de *frames*, evaluarse en instantes originales o intermedios, y servir como base para los experimentos de reconstrucción, interpolación y reducción presentados en los capítulos siguientes.

CAPÍTULO V

Experimentos y resultados de la representación de video

5.1 Base monomial frente a base de Chebyshev (Experimento 1)

El primer experimento tuvo como objetivo determinar qué base polinómica era más adecuada para representar la evolución temporal de los atributos de las gaussianas. Para ello, se comparó una base monomial con una base formada por polinomios de Chebyshev.

En ambos casos se mantuvo la misma idea general: cada atributo de las gaussianas se expresó como una combinación de funciones del tiempo cuyos coeficientes eran ajustados durante el entrenamiento. La diferencia se encontraba únicamente en las funciones utilizadas como base.

En la representación monomial, un atributo temporal se expresa como

$$p(t) = a_0 + a_1 t + a_2 t^2 + \cdots + a_K t^K, \tag{V.1}$$

mientras que con Chebyshev se utiliza

$$p(t) = \sum_{k=0}^{K} a_k T_k\left(\tau(t)\right), \tag{V.2}$$

donde $\tau(t)$ corresponde al tiempo normalizado al intervalo $[-1, 1]$.

La hipótesis inicial fue que ambas bases podían representar cambios temporales, pero que la base monomial presentaría mayores dificultades al aumentar el grado del polinomio. Las potencias elevadas del tiempo pueden producir valores

con escalas muy diferentes y dificultar la actualización de los coeficientes durante el entrenamiento.

En cambio, los polinomios de Chebyshev mantienen un comportamiento más controlado dentro del intervalo normalizado. Por esta razón, se esperaba que permitieran una optimización más estable y un mejor aprovechamiento de los términos de mayor grado.

Para realizar una comparación justa, ambas configuraciones utilizaron el mismo clip, la misma cantidad de gaussianas, los mismos grados temporales, la misma cantidad de épocas, la misma función de pérdida y los mismos parámetros de entrenamiento. La diferencia principal fue la base temporal utilizada para representar los atributos de las gaussianas.

En ambos casos se utilizaron 8000 gaussianas iniciales y 360 épocas de entrenamiento. Los grados empleados fueron $K_\mu = 30$, $K_\alpha = 20$, $K_{color} = 15$, $K_{scale} = 8$, $K_\theta = 4$ y $K_{depth} = 4$. Además, se utilizó la pérdida *motion*, con $\lambda_{\text{DSSIM}} = 0.25$ y $\lambda_{\text{motion}} = 2.0$.

Los resultados se presentan en la Tabla 5.1.

Tabla 5.1: Comparación entre base monomial y base de Chebyshev.

Base temporal	PSNR prom.	PSNR mín.	SSIM prom.	PSNR temp. prom.	PSNR std.
Monomial	23.16	20.19	0.750	30.61	2.61
Chebyshev	**29.95**	**27.70**	**0.926**	**34.04**	**1.09**

La Figura 5.1 muestra una comparación visual entre la reconstrucción obtenida con la base de Chebyshev y la base monomial.



FIGURA 5.1: Comparación visual entre la reconstrucción con base de Chebyshev (izquierda) y base monomial (derecha).

Video demostrativo del Experimento 1: la comparación en video entre la base de Chebyshev y la base monomial se encuentra disponible en: https://www.youtube.com/watch?v=doHav1asWXc

La base de Chebyshev obtuvo mejores resultados que la base monomial en todas las métricas principales. El PSNR promedio aumentó de 23.16 dB a 29.95 dB, mientras que el SSIM promedio pasó de 0.750 a 0.926. Además, el PSNR mínimo también mejoró de 20.19 dB a 27.70 dB, lo que indica que la mejora no ocurrió solo en algunos *frames*, sino también en los casos más difíciles.

Otro resultado importante es la reducción de la desviación estándar del PSNR, que pasó de 2.61 en la base monomial a 1.09 en la base de Chebyshev. Esto sugiere que Chebyshev no solo logró una mejor reconstrucción promedio, sino también un comportamiento más estable a lo largo del video.

Estos resultados respaldan la elección de Chebyshev como base temporal del modelo. En comparación con la base monomial, Chebyshev permitió representar con mayor calidad y estabilidad la evolución de los atributos de las gaussianas.

5.2 Grados del polinomio y número de gaussianas (Experimento 2)

El segundo experimento tuvo como objetivo estudiar cómo influyen dos elementos principales en la capacidad del modelo: el número de gaussianas y el grado de las funciones temporales.

El número de gaussianas determina cuántas regiones pueden utilizarse para representar el contenido visual. Una cantidad reducida puede ser suficiente para describir fondos o zonas con pocos detalles, pero puede dificultar la representación de objetos pequeños, texturas o movimientos complejos. Por otro lado, aumentar demasiado esta cantidad incrementa el uso de memoria y el tiempo necesario para el entrenamiento.

El grado de los polinomios determina qué tan compleja puede ser la evolución temporal de cada atributo. Una función de grado bajo representa cambios suaves, mientras que una función de mayor grado puede describir movimientos y variaciones más complejas. Sin embargo, incrementar el grado también aumenta el número de coeficientes que deben aprenderse y puede dificultar la optimización.

No todos los atributos necesitan el mismo grado. La posición y la opacidad pueden presentar cambios frecuentes durante el video, mientras que la escala, la orientación o la profundidad pueden variar de forma más suave. Por esta razón, se evaluaron configuraciones con grados diferentes para cada grupo de atributos.

Para comparar las configuraciones se mantuvieron constantes el video, la función de pérdida, el número de épocas y los demás parámetros de entrenamiento. Se registraron las métricas de calidad, el tiempo de entrenamiento y el uso de memoria.

Tabla 5.2: Resultados del Experimento 2 al variar el número de gaussianas y los grados temporales.

Configuración	Gaussianas	Grados principales	PSNR	SSIM	PSNR temp.
Base	8k	30/20/15/8/4/4	29.95	0.926	34.04
Más gaussianas	12k	30/20/15/8/4/4	30.92	0.938	34.76
Más gaussianas	16k	30/20/15/8/4/4	**31.91**	**0.948**	**35.51**
Más grados	8k	36/26/20/10/6/4	30.29	0.930	34.46
Más grados	8k	45/32/26/14/8/6	31.09	0.940	35.28

La Figura 5.2 muestra el efecto visual de aumentar la cantidad de gaussianas en la reconstrucción.

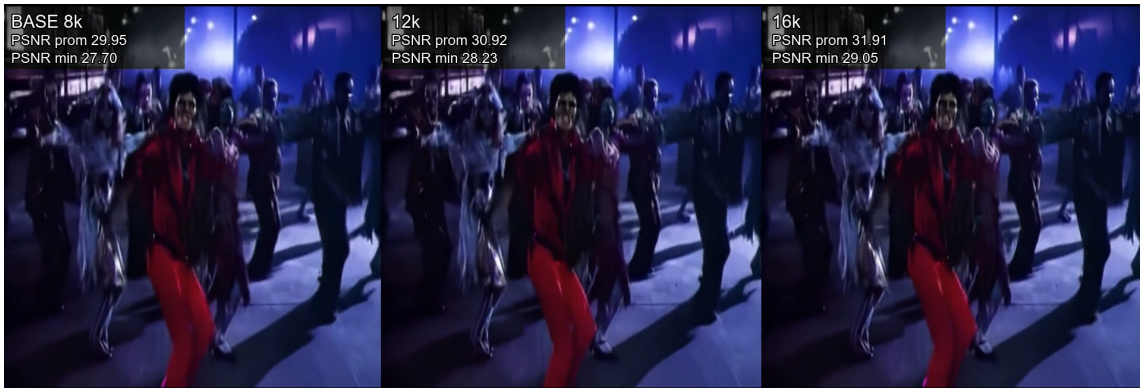


FIGURA 5.2: Comparación visual del efecto de aumentar el número de gaussianas en la reconstrucción del video.

Video demostrativo del Experimento 2: la comparación visual al variar la cantidad de gaussianas y los grados temporales se encuentra disponible en: https://www.youtube.com/watch?v=vqdti79Xc4A

El PSNR temporal mide qué tan bien el modelo reproduce los cambios entre *frames* consecutivos. A diferencia del PSNR normal, que evalúa cada *frame* por separado, el PSNR temporal ayuda a ver si el movimiento del video reconstruido se parece al movimiento del video original.

Esta métrica también se expresa en decibeles (dB) y, en general, mientras más alto sea el valor, mejor es la reconstrucción temporal. Como referencia, valores cercanos a 30 dB ya indican una reconstrucción razonable, mientras que valores alrededor de 40 dB o más suelen significar que las diferencias visuales son poco perceptibles. De todas formas, esto puede variar según el tipo de video y qué tan rápido sea el movimiento.

Todas las configuraciones se evaluaron sobre el mismo clip, con 120 *frames*, resolución de 288×512, base temporal de Chebyshev y la misma función de pérdida *motion*. La configuración base utilizó 8000 gaussianas y grados temporales 30/20/15/8/4/4 para posición, opacidad, color, escala, rotación y profundidad, respectivamente.

En la primera comparación se mantuvieron los mismos grados temporales y se aumentó únicamente el número de gaussianas. Al pasar de 8000 a 12000 gaussianas,

el PSNR promedio aumentó de 29.95 dB a 30.92 dB. Al incrementar la cantidad a 16000 gaussianas, el PSNR alcanzó 31.91 dB. Este comportamiento muestra que una mayor cantidad de gaussianas permite representar más detalles visuales del video, lo que se refleja también en el aumento del SSIM de 0.926 a 0.948.

En la segunda comparación se mantuvo fijo el número de gaussianas en 8000 y se aumentaron los grados temporales. Con grados 36/26/20/10/6/4, el PSNR subió a 30.29 dB, mientras que con grados 45/32/26/14/8/6 alcanzó 31.09 dB. Esto indica que aumentar la capacidad temporal de los atributos también mejora la reconstrucción, aunque la mejora fue más moderada que la obtenida al aumentar el número de gaussianas.

Estos resultados sugieren que la calidad de reconstrucción depende tanto de la cantidad de primitivas disponibles como de la capacidad temporal asignada a sus atributos. Más gaussianas permiten cubrir mejor la estructura espacial del video, mientras que grados más altos permiten describir cambios temporales más complejos. Sin embargo, ambos incrementos también elevan el número de parámetros y el costo computacional.

Debido a que estos entrenamientos se realizaron con una cantidad limitada de épocas, los resultados deben interpretarse como una comparación inicial de tendencias. Es razonable esperar que entrenamientos más prolongados permitan alcanzar valores mayores de PSNR, especialmente en las configuraciones con más gaussianas o mayor capacidad temporal. Aun así, los resultados obtenidos ya muestran una tendencia clara: aumentar la capacidad del modelo mejora la reconstrucción del video.

En conjunto, el Experimento 2 muestra que MBB-GS escala de forma positiva al incrementar la capacidad del modelo, tanto mediante más gaussianas como mediante mayores grados temporales.

5.3 Función de pérdida: ablación por fases y peso por movimiento (Experimento 3)

Una vez definida la estructura general del modelo, se estudió la influencia de la función de pérdida. Esta función determina qué errores reciben mayor importancia durante el entrenamiento y, por tanto, afecta directamente la forma en que las gaussianas aprenden a representar el video.

La evaluación se dividió en dos fases. En la primera se compararon distintas formas de calcular el error dentro de cada *frame*. En la segunda se analizó cómo combinar los errores correspondientes a varios *frames* procesados durante una misma iteración.

Fase 1: comparación de funciones de pérdida

Todas las configuraciones se entrenaron sobre un video de 600 *frames*, con resolución de 720p, 100 000 gaussianas y 800 épocas. Se mantuvieron constantes los demás parámetros para que la principal diferencia fuera la función de pérdida utilizada.

Las variantes evaluadas fueron las siguientes:

- ***Baseline*:** combina el error absoluto entre píxeles con una medida de similitud estructural.
- ***Edge*:** añade mayor importancia a los bordes y contornos presentes en la imagen.
- ***Temporal*:** compara los cambios entre *frames* consecutivos de la reconstrucción con los cambios del video original.
- ***Motion*:** utiliza la diferencia entre dos *frames* consecutivos del video original para localizar las zonas con movimiento y dar mayor importancia al error producido en estas regiones.

En la variante *motion*, primero se construye una máscara de movimiento a partir de dos *frames* consecutivos del video original:

$$M_t = \text{normalizar}\left(|I_t - I_{t-1}|\right), \tag{V.3}$$

donde I_t representa el *frame* actual e I_{t-1} el *frame* anterior. A partir de esta máscara se obtiene un peso por píxel:

$$W_t = 1 + \lambda_{\text{motion}} M_t. \tag{V.4}$$

Las regiones que presentan pocos cambios reciben un peso cercano a uno, mientras que las regiones dinámicas reciben un peso mayor. La pérdida absoluta ponderada se calcula como

$$\mathcal{L}_{\text{motion}} = \frac{\sum_{\mathbf{p}} W_t(\mathbf{p}) \left|\hat{I}_t(\mathbf{p}) - I_t(\mathbf{p})\right|}{\sum_{\mathbf{p}} W_t(\mathbf{p})}, \tag{V.5}$$

donde $\hat{I}_t$ es el *frame* reconstruido. En la implementación, la máscara se calcula únicamente a partir del video original y se utiliza como una guía fija durante el cálculo de los gradientes.

La función final combina este error con DSSIM:

$$\mathcal{L} = 0.75\,\mathcal{L}_{\text{motion}} + 0.25\,\mathcal{L}_{\text{DSSIM}}. \tag{V.6}$$

Los resultados de la primera fase se presentan en la Tabla 5.3.

Tabla 5.3: Resultados de la ablación de funciones de pérdida.

Pérdida	PSNR	PSNR mín.	PSNR p5	SSIM	LPIPS	PSNR temp.
Baseline	31.83	28.10	29.35	0.962	0.0594	35.17
Edge	31.88	28.05	29.30	0.963	0.0574	35.23
Motion	**33.24**	**30.32**	**31.12**	**0.965**	**0.0483**	**36.88**
Temporal	31.93	28.32	29.42	0.963	0.0588	35.32

En PSNR, SSIM y PSNR temporal, un valor mayor indica una mejor reconstrucción. En LPIPS ocurre lo contrario: un valor menor indica una mayor similitud perceptual con el video original.

La variante *motion* obtuvo los mejores resultados en todas las métricas. Alcanzó un PSNR promedio de 33.24 dB, frente a 31.83 dB del *baseline*. También presentó el mayor PSNR temporal y el menor LPIPS.

Estos resultados indican que dar mayor importancia a las regiones dinámicas ayuda al modelo a representar mejor los movimientos sin perjudicar la calidad general del video. Las variantes *l1_mse* y *combo* también formaban parte de la evaluación, pero sus entrenamientos no terminaron debido a fallas en el entorno de ejecución, por lo que no se incluyeron en la tabla.

Fase 2: combinación del error entre frames

Después de seleccionar la variante *motion*, se estudió cómo combinar el error de los distintos *frames* procesados dentro de una misma iteración. Para ello, se utilizó un exponente q que controla cuánto énfasis se coloca sobre los *frames* con mayor error.

Cuando $q = 1$, los errores se promedian de manera uniforme. Al aumentar q, los *frames* más difíciles reciben una importancia progresivamente mayor.

Los resultados se muestran en la Tabla 5.4.

Tabla 5.4: Resultados al variar el exponente utilizado para combinar el error entre frames.

q	PSNR	PSNR mín.	SSIM	LPIPS	PSNR temp.
1	**33.24**	**30.32**	**0.965**	**0.048**	**36.90**
2	29.77	27.43	0.942	0.099	33.55
4	21.19	16.84	0.859	0.336	30.33
8	14.70	13.37	0.517	0.821	30.03

Los resultados muestran una disminución continua de la calidad al aumentar el valor de q. El promedio convencional, correspondiente a $q = 1$, obtuvo los mejores resultados, mientras que $q = 8$ redujo el PSNR hasta 14.70 dB.

Una posible explicación es que, al utilizar valores elevados de q, el entrenamiento concentra gran parte de la actualización en unos pocos *frames* con errores altos. Esto produce gradientes menos equilibrados y dificulta el aprendizaje de los coeficientes temporales, especialmente los de mayor grado.

A partir de ambas fases se seleccionó la pérdida *motion* con un promedio uniforme entre *frames*. Esta configuración fue utilizada como base para los experimentos posteriores del modelo de video.

CAPÍTULO VI

Aplicaciones del framework de video

6.1 Reconstrucción/interpolación de frames y cámara lenta

Una de las ventajas de representar el video mediante gaussianas 2D temporales es que el modelo no queda limitado únicamente a los *frames* originales del video. Como los atributos de cada gaussiana se definen como funciones del tiempo, el modelo puede evaluarse también en instantes intermedios. Esto permite generar nuevos *frames* entre dos *frames* reales y producir una versión en cámara lenta del video.

En una representación tradicional, el video está formado por una secuencia fija de imágenes. Si se desea aumentar la cantidad de *frames*, normalmente se requiere un método externo de interpolación. En cambio, en este framework, la interpolación se obtiene evaluando el mismo modelo entrenado en una mayor cantidad de posiciones temporales. Por ejemplo, si el modelo fue entrenado con un video a 60 FPS, se puede generar una versión a 480 FPS evaluando más puntos intermedios en el tiempo.

El proceso utilizado para generar la interpolación fue el siguiente. Primero, se entrenó el modelo sobre el clip original. Luego, a partir del *checkpoint* final, se construyó una nueva matriz temporal de Chebyshev con una mayor cantidad de *frames*. Finalmente, el modelo fue evaluado en estos nuevos instantes y se renderizaron los *frames* interpolados. Este procedimiento no entrena un nuevo interpolador, sino que reutiliza la representación temporal aprendida por el modelo.

En el experimento de interpolación se utilizó un clip de 180 *frames* a resolución 720p. El modelo fue entrenado con 90000 gaussianas, base temporal de Chebyshev, pérdida *motion* y 2200 épocas. Los grados temporales usados fueron

26/24/12/5/3/2 para posición, opacidad, color, escala, rotación y profundidad, respectivamente. Además, se utilizó una regularización temporal suave con $\beta = 4\times10^{-8}$.

La reconstrucción del clip original obtuvo un PSNR promedio de 40.97 dB, un SSIM promedio de 0.985 y un LPIPS promedio de 0.028. Además, el PSNR temporal promedio fue de 43.78 dB, lo que indica que el modelo logró reproducir de forma coherente los cambios entre *frames* consecutivos.

Tabla 6.1: Métricas de reconstrucción del modelo usado para interpolación temporal.

Métrica	Valor
Gaussianas	90000
Frames originales	180
Resolución	720p
PSNR promedio	40.97 dB
PSNR mínimo	36.83 dB
SSIM promedio	0.985
LPIPS promedio	0.028
PSNR temporal promedio	43.78 dB

A partir del mismo modelo se generaron versiones interpoladas a mayor tasa de *frames*.Estas versiones permiten observar el movimiento de forma más lenta y continua. La idea principal es que la cámara lenta no se obtiene duplicando *frames*, sino evaluando la función temporal aprendida en instantes intermedios.

Videos demostrativos de interpolación temporal:

- Interpolación 1: https://www.youtube.com/watch?v=_nBj2Qm1U1g
- Interpolación 2: https://www.youtube.com/watch?v=v-bB8F2yiG8

Aunque los resultados muestran que la representación temporal permite generar *frames* intermedios, también se observaron algunas limitaciones. En zonas con

movimiento rápido pueden aparecer artefactos visuales, como rastros, deformaciones o zonas ligeramente borrosas. Esto ocurre porque el modelo puede aprender a reconstruir bien los *frames* conocidos, pero no siempre garantiza que la trayectoria intermedia sea visualmente perfecta. Aun así, la interpolación demuestra una ventaja importante del enfoque: el video puede ser tratado como una señal continua en el tiempo.

6.2 Poda de gaussianas y reducción del modelo

Otra aplicación del framework es la reducción del modelo. Como la representación está formada por gaussianas explícitas, es posible analizar cuáles de ellas aportan más a la reconstrucción y cuáles podrían eliminarse sin afectar demasiado la calidad visual. Esta idea permite explorar estrategias de poda (*pruning*) y, posteriormente, cuantización de parámetros.

La poda busca reducir la cantidad de gaussianas del modelo. En lugar de usar todas las gaussianas aprendidas durante el entrenamiento, se eliminan aquellas que tienen menor contribución visual según los criterios definidos por el framework. Después de aplicar la poda, el video se vuelve a renderizar y se compara contra la reconstrucción completa. Si la pérdida de calidad es pequeña, la poda se conserva; si la calidad cae demasiado, se mantiene una versión menos agresiva o el modelo original.

Para este experimento se utilizó un video de 750 *frames* a resolución 720p. El modelo fue entrenado con 150000 gaussianas iniciales durante 1600 épocas. Los grados temporales usados fueron 100/80/30/12/6/4 para posición, opacidad, color, escala, rotación y profundidad, respectivamente. Al igual que en los experimentos anteriores, se utilizó la pérdida *motion*, con $\lambda_{\text{DSSIM}} = 0.25$ y $\lambda_{\text{motion}} = 2.0$.

Antes de aplicar reducción, la reconstrucción completa obtuvo un PSNR promedio de 35.94 dB, un SSIM promedio de 0.970 y un LPIPS promedio de 0.048.

El PSNR temporal promedio fue de 39.59 dB, lo que indica una buena consistencia temporal en la reconstrucción del movimiento.

Tabla 6.2: Métricas de la reconstrucción completa antes de la reducción del modelo.

Métrica	Valor
Gaussianas iniciales	150000
Frames	750
Resolución	720p
PSNR promedio	35.94 dB
PSNR mínimo	27.40 dB
PSNR p5	30.71 dB
SSIM promedio	0.970
LPIPS promedio	0.048
PSNR temporal promedio	39.59 dB

En una de las pruebas se aplicó una poda adaptativa del 20 % de las gaussianas. Esto significa que se eliminaron aproximadamente 30000 gaussianas del modelo original, dejando una reconstrucción con 120000 gaussianas. Esta selección se realizó sobre un modelo inicial de 150000 gaussianas.

Después de la poda, se puede aplicar una etapa de cuantización. En esta etapa, los parámetros del modelo se convierten a una representación más compacta, por ejemplo usando enteros de 16 bits (`uint16`) en lugar de valores en punto flotante. La cuantización permite reducir el tamaño de almacenamiento del modelo, aunque puede introducir una pequeña pérdida de precisión. Por esta razón, la cuantización debe evaluarse comparando nuevamente la calidad de los *frames* reconstruidos.

El flujo general de reducción utilizado por el framework puede resumirse así:

1. Entrenar el modelo completo con todas las gaussianas.
2. Calcular métricas de reconstrucción del modelo completo.
3. Seleccionar gaussianas candidatas para poda.

4. Renderizar el video con el subconjunto reducido.

5. Comparar la calidad contra la reconstrucción completa.

6. Aceptar la poda solo si la pérdida de calidad es pequeña.

7. Cuantizar los parámetros del modelo reducido.

Tabla 6.3: Comparación de tamaño entre el modelo completo y la versión reducida cuantizada.

Versión	Gaussianas	Tamaño aprox.	PSNR vs. reconstrucción completa
Modelo completo	150000	236 MB	Referencia
Poda adaptativa 20 % + `uint16`	120000	85 MB	63 dB

La versión reducida y cuantizada disminuyó el tamaño aproximado del modelo de 236 MB a 85 MB. Esto representa una reducción cercana al 64 % respecto al modelo completo. Además, el PSNR de la versión cuantizada fue de aproximadamente 63 dB al compararla contra la reconstrucción completa de 150000 gaussianas.

Es importante aclarar que, en esta prueba, la métrica de cuantización se calculó usando como referencia la reconstrucción completa, no directamente el video original. Por ello, este valor indica qué tan cercana es la versión reducida y cuantizada respecto a la reconstrucción completa usada como referencia.

En términos prácticos, esta prueba muestra que fue posible reducir el tamaño del modelo en más de la mitad, manteniendo una salida visual muy cercana a la reconstrucción completa. Esto sugiere que no todas las gaussianas aportan de la misma manera a la reconstrucción del video. Algunas son más importantes porque representan regiones con mayor información visual, movimiento o detalle, mientras que otras tienen un aporte menor y pueden eliminarse con poco impacto visual.

Video demostrativo de reducción del modelo: la comparación entre la reconstrucción completa, la versión podada y la versión cuantizada se encuentra disponible en: https://www.youtube.com/watch?v=j5ycJokEZg4

Este resultado debe interpretarse como una exploración inicial de reducción del modelo. Aunque la poda y la cuantización permiten obtener una representación más compacta, todavía no constituyen un esquema completo de compresión de video ni un códec competitivo frente a métodos tradicionales. Para llegar a ese punto, sería necesario estudiar aspectos adicionales como la codificación eficiente de parámetros, el almacenamiento, la transmisión, la decodificación y la comparación directa con formatos de video convencionales. Aun así, los resultados muestran que una representación explícita con gaussianas permite analizar qué partes del modelo son más relevantes y abre la posibilidad de desarrollar estrategias de compresión más eficientes en trabajos futuros.

CAPÍTULO VII

Representación de audio con átomos de Gabor

En los capítulos anteriores se presentó la representación de video mediante gaussianas temporales. A partir de esa idea, se buscó explorar si un principio similar podía aplicarse también al audio. Sin embargo, el audio tiene características distintas al video: es una señal altamente oscilatoria, depende de la fase para reconstruirse correctamente y puede cambiar muy rápido incluso en intervalos pequeños de tiempo.

Por esta razón, esta parte del trabajo se entiende como una extensión exploratoria del framework principal. El objetivo no fue resolver por completo la representación de audio, sino probar si las primitivas explícitas también podían servir para aproximar señales sonoras. Primero se intentó adaptar directamente la idea usada en video mediante gaussianas y polinomios de Chebyshev. Luego, al observar las limitaciones de ese enfoque, se probó una representación basada en átomos de Gabor, la cual resultó más adecuada para reconstruir la forma de onda.

7.1 Intento con Chebyshev sobre el espectrograma y sus limitaciones (Experimento 4)

La primera idea para representar audio no fue usar átomos de Gabor. El punto de partida fue reutilizar el código y la lógica que ya se habían desarrollado para video. En video, los atributos de las gaussianas cambian en el tiempo mediante polinomios de Chebyshev. Por ello, se intentó aplicar una idea parecida al audio: usar gaussianas cuyos parámetros variaran temporalmente.

En la práctica, este primer enfoque no trabajaba directamente sobre la forma de onda, sino sobre la magnitud del espectrograma. Para un audio de entrada, primero se calculaba la STFT y luego se tomaba la magnitud logarítmica normalizada

como objetivo de entrenamiento. El modelo intentaba reconstruir esta representación tiempo-frecuencia mediante gaussianas ubicadas sobre el eje de frecuencia. Cada gaussiana tenía un centro frecuencial, un ancho y una amplitud, y estos parámetros cambiaban en el tiempo usando coeficientes de Chebyshev.

De manera simplificada, el espectrograma reconstruido podía expresarse como una suma de gaussianas en frecuencia:

$$\hat{S}(t,f) = 1 - \exp\left(-\sum_{i=1}^{N} A_i(t) \exp\left(-\frac{(f-\mu_{f,i}(t))^2}{2\sigma_{f,i}(t)^2}\right)\right), \tag{VII.1}$$

donde $\mu_{f,i}(t)$ representa el centro de la gaussiana en frecuencia, $\sigma_{f,i}(t)$ controla su ancho y $A_i(t)$ define su amplitud. Cada uno de estos parámetros era una función temporal representada mediante polinomios de Chebyshev.

Esta formulación era atractiva porque aprovechaba parte de la estructura ya usada en video. Además, permitía mantener la idea de primitivas gaussianas con parámetros dependientes del tiempo. Sin embargo, al llevarla al audio aparecieron varias limitaciones importantes.

La primera limitación fue que el modelo solo aprendía magnitud espectral. Es decir, no aprendía la fase de la señal. Esto es un problema fuerte en audio, porque para volver desde un espectrograma a una forma de onda se necesita información de fase. En nuestras pruebas, se podían generar dos salidas: una usando la fase original del audio y otra estimando la fase mediante Griffin-Lim [12]. La primera opción no era una reconstrucción completamente autónoma, porque dependía de información del audio original. La segunda opción sí era más independiente, pero agregaba una etapa iterativa adicional y podía introducir artefactos.

La segunda limitación fue que la magnitud tampoco se reconstruía suficientemente bien. Aunque el modelo podía capturar parte de la energía general del espectrograma, tenía dificultades para reproducir detalles finos del audio, especialmente en regiones con componentes armónicas, transientes o cambios rápidos. En

consecuencia, el audio reconstruido podía sonar reconocible, pero no alcanzaba una fidelidad adecuada.

La tercera limitación fue el tiempo de entrenamiento. Aunque se implementaron versiones aceleradas en CUDA para el render y para algunas pérdidas, el entrenamiento seguía siendo costoso porque debía operar sobre una malla tiempo-frecuencia completa. En corridas exploratorias, este enfoque podía demorar entre 20 y 30 minutos y aun así producir valores de SNR bajos, aproximadamente entre 2 dB y 6 dB. Estos resultados no eran suficientes para sostener esta línea como la representación final de audio.

En resumen, este experimento fue útil porque mostró que una adaptación directa del modelo de video al audio no era suficiente. El problema no era solo de implementación o velocidad, sino también de representación: al modelar únicamente magnitud espectral, se dejaba fuera la fase, que es una parte esencial de la señal sonora. Por ello, se decidió buscar una representación que trabajara directamente sobre la forma de onda y que pudiera modelar mejor las oscilaciones propias del audio.

7.2 Modelo de átomos de Gabor (Experimento 5)

Después de las limitaciones observadas en el enfoque anterior, se probó una representación basada en átomos de Gabor. Esta alternativa resultó más natural para audio, ya que un átomo de Gabor combina una envolvente gaussiana con una oscilación sinusoidal [3]. Esta combinación permite representar componentes localizadas en el tiempo y, al mismo tiempo, capturar oscilaciones asociadas a distintas frecuencias.

La señal reconstruida se define como una suma de átomos:

$$\hat{x}(t) = \sum_{i=1}^{N} A_i \exp\left(-\frac{(t-\mu_i)^2}{2\sigma_i^2}\right) \cos\left(2\pi f_i(t-\mu_i) + \phi_i\right), \tag{VII.2}$$

donde μ_i representa la posición temporal del átomo, σ_i controla su duración, A_i es su amplitud, f_i es la frecuencia y ϕ_i es la fase.

La diferencia más importante frente al enfoque anterior es que aquí la reconstrucción se realiza directamente en el dominio temporal. Ya no se entrena un espectrograma para luego convertirlo a audio. En cambio, el modelo genera directamente una forma de onda $\hat{x}(t)$. Esto elimina la necesidad de recuperar fase con Griffin-Lim, porque la fase ya forma parte de los parámetros aprendibles de cada átomo.

Esta representación también se relaciona con métodos de descomposición atómica, como Matching Pursuit, donde una señal se aproxima mediante la combinación de elementos de un diccionario [4]. En nuestro caso, la diferencia es que los átomos no se seleccionan de un diccionario fijo, sino que sus parámetros se optimizan mediante gradiente.

Para hacer viable el entrenamiento, se implementó un renderizador CUDA para los átomos de Gabor. Cada átomo solo contribuye de forma significativa dentro de una ventana local controlada por k_σ, lo cual evita evaluar cada átomo sobre toda la señal cuando su contribución fuera de esa región es prácticamente nula. El forward genera la forma de onda reconstruida, mientras que el backward calcula los gradientes respecto a μ_i, σ_i, A_i, f_i y ϕ_i.

En comparación con el intento basado en Chebyshev y espectrogramas, el modelo de Gabor presentó tres ventajas claras. Primero, reconstruye directamente la señal de audio. Segundo, incorpora la fase como parte del modelo. Tercero, obtuvo mejores resultados en menor tiempo. Por estas razones, se eligió Gabor como la representación final de audio dentro del alcance de esta tesis.

Es importante aclarar que esta línea no fue explorada de manera exhaustiva. El eje principal de la tesis sigue siendo la representación de video mediante gaussianas temporales. El audio se trabajó como una extensión complementaria para evaluar si el mismo principio general de primitivas optimizables podía trasladarse a

otra modalidad. Aun así, los resultados obtenidos con Gabor fueron suficientemente buenos como para considerarlo una dirección prometedora.

7.3 Ablaciones: pérdida, inicialización y mono/estéreo (Experimento 6)

Una vez adoptado el modelo de átomos de Gabor, se realizaron pruebas para entender qué decisiones afectaban más la calidad final de la reconstrucción. Estas pruebas no buscaron cubrir todo el espacio posible de configuraciones, sino identificar qué elementos eran más importantes para obtener un resultado estable y de buena calidad.

7.3.1 Pérdida de entrenamiento

En audio, comparar únicamente la forma de onda muestra a muestra puede ser limitado. Dos señales pueden sonar parecidas y aun así presentar diferencias importantes a nivel de muestras debido a pequeños desfases. Por esta razón, además de una pérdida sobre waveform, se utilizaron términos en el dominio tiempo-frecuencia.

La pérdida final combinó tres componentes principales. El primero fue un término directo sobre la forma de onda, que ayuda a mantener la señal cerca del objetivo en el dominio temporal. El segundo fue una pérdida multi-resolution STFT, que compara la magnitud espectral de la señal en varias resoluciones tiempo-frecuencia. Este tipo de pérdida se utiliza en modelos modernos de generación de audio porque permite evaluar la estructura espectral a distintas escalas [14]. El tercer componente fue un término asociado a SI-SDR, una métrica común para evaluar reconstrucción y separación de señales de audio [15].

De forma general, la pérdida utilizada puede escribirse como

$$\mathcal{L} = \lambda_{\text{wave}}\mathcal{L}_{\text{wave}} + \lambda_{\text{MR-STFT}}\mathcal{L}_{\text{MR-STFT}} + \lambda_{\text{SI-SDR}}\mathcal{L}_{\text{SI-SDR}}. \tag{VII.3}$$

Esta combinación fue más efectiva que usar únicamente error muestra a muestra. La parte MR-STFT ayudó a preservar mejor el contenido espectral, mientras que el término SI-SDR ayudó a mejorar la fidelidad global de la reconstrucción. Este enfoque también está alineado con la idea de integrar procesamiento de señales y optimización diferenciable, como ocurre en trabajos modernos de audio diferenciable [13].

7.3.2 Inicialización

La inicialización también tuvo un papel importante. Una inicialización completamente aleatoria puede funcionar, pero en audio hace que el modelo empiece sin una pista clara de dónde están las regiones importantes de la señal. Por ello, se probó una inicialización basada en energía.

En esta inicialización, los átomos se colocan inicialmente en regiones tiempo-frecuencia con mayor energía. La idea no es resolver el problema desde el inicio, sino darle al modelo una mejor posición de partida. Esto fue especialmente útil en señales musicales, donde existen eventos con mucha energía, patrones armónicos y transientes que conviene cubrir desde las primeras épocas.

En la práctica, esta inicialización permitió que los átomos se distribuyeran mejor sobre la señal y que el entrenamiento convergiera hacia reconstrucciones de mayor calidad.

7.3.3 Mono y estéreo

También se probaron configuraciones mono y estéreo. En mono, el problema consiste en aproximar una única señal. En estéreo, en cambio, existen dos canales y se debe preservar no solo la calidad sonora, sino también la imagen estéreo.

Para el caso estéreo se trabajó en el dominio Mid-Side. A partir de los canales izquierdo L y derecho R, se definen

$$M = \frac{L + R}{2}, \tag{VII.4}$$

$$S = \frac{L - R}{2}. \tag{VII.5}$$

El canal Mid concentra gran parte de la información común entre los dos canales, mientras que el canal Side contiene la diferencia entre ellos. Esta separación permite asignar distinta capacidad a cada componente. Por ejemplo, se puede usar más átomos para Mid y menos para Side, o aumentar la cantidad de átomos en Side cuando se quiere preservar mejor la información estéreo.

Para volver al dominio izquierdo-derecho, se usa

$$L = M + S, \tag{VII.6}$$

$$R = M - S. \tag{VII.7}$$

Este esquema resultó conveniente porque permitió controlar de forma más clara cuánta capacidad se asignaba a la parte principal del audio y cuánta a la información estéreo.

7.3.4 Ejemplo comparativo en un fragmento musical

Para mostrar de manera concreta cómo cambia la reconstrucción al modificar la configuración, se tomó como ejemplo un fragmento estéreo de 9 segundos de la canción *Rock That Body* de Black Eyed Peas, correspondiente aproximadamente al intervalo entre los segundos 31 y 40. Sobre este mismo fragmento se entrenaron dos configuraciones en dominio Mid-Side: una configuración reducida y una configuración final de mayor capacidad.

La configuración reducida utilizó 24 000 átomos en total, distribuidos en 16 000 para Mid y 8 000 para Side, con 800 épocas de entrenamiento. En cambio, la configuración final utilizó 160 000 átomos, distribuidos en 96 000 para Mid y 64 000 para Side, con 6000 épocas. La Tabla 7.1 resume las métricas obtenidas en ambas pruebas.

Métrica	**Config. reducida**	**Config. final**
Número total de átomos	24 000	160 000
Distribución Mid/Side	16k / 8k	96k / 64k
Épocas	800	6000
Tiempo de entrenamiento	0.35 min	6.33 min
SNR estéreo	6.88 dB	27.78 dB
SI-SDR estéreo	5.89 dB	27.77 dB
PSNR estéreo	23.34 dB	44.24 dB
LSD promedio	9.10 dB	4.57 dB
Mel-L1 promedio	0.9089	0.1985
MR-STFT promedio	2.4352	1.8532
SNR Mid	7.10 dB	28.50 dB
SNR Side	2.67 dB	20.01 dB

Tabla 7.1: Comparación de métricas entre una configuración reducida y una configuración final de átomos de Gabor sobre un fragmento estéreo de 9 segundos.

Como se observa en la Tabla 7.1, la configuración reducida logra reconstruir un audio reconocible, pero con una calidad todavía baja. El SNR estéreo alcanza 6.88 dB y el SI-SDR estéreo llega a 5.89 dB. Además, el SNR del componente Side es de solo 2.67 dB, lo que indica que la información estéreo se conserva de forma limitada.

En cambio, la configuración final mejora de manera clara todas las métricas principales. El SNR estéreo aumenta hasta 27.78 dB, el SI-SDR estéreo llega a 27.77 dB y el PSNR estéreo sube a 44.24 dB. Además, el LSD promedio disminuye de 9.10 dB a 4.57 dB, lo que indica que la estructura espectral del audio reconstruido se parece más a la del audio original. También es importante notar que el SNR del componente Side aumenta de 2.67 dB a 20.01 dB, por lo que la configuración final preserva mejor la imagen estéreo.

Esta comparación muestra que el modelo de Gabor es sensible a la capacidad y al tiempo de optimización. Con pocos átomos y pocas épocas, el entrenamiento

es muy rápido, pero la calidad es limitada. En cambio, al aumentar la cantidad de átomos, entrenar durante más épocas y asignar mayor capacidad al componente Side, la reconstrucción mejora de forma importante tanto en waveform como en contenido espectral.

La Figura 7.1 muestra la comparación entre la forma de onda original y la reconstrucción obtenida con la configuración reducida. Se observa que el modelo logra seguir la estructura general de la señal, pero todavía presenta diferencias visibles respecto al audio objetivo.

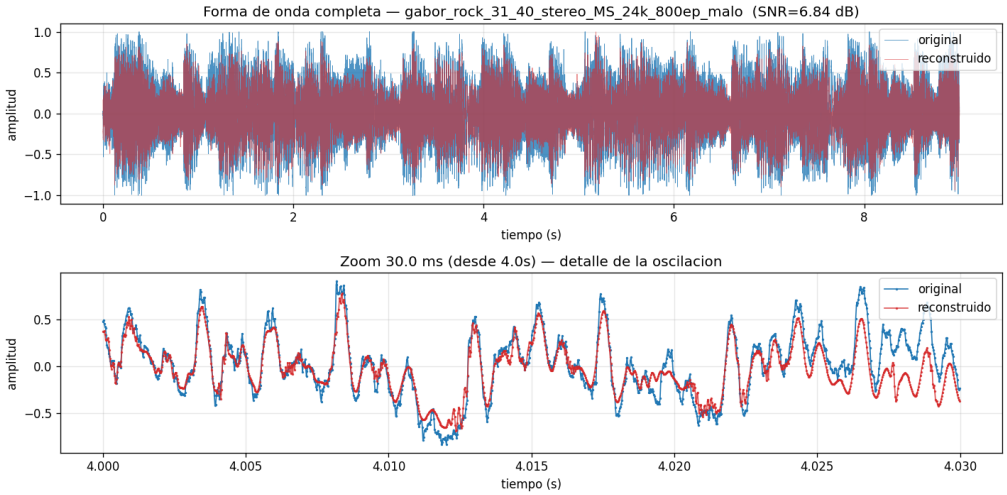


FIGURA 7.1: Comparación de la forma de onda original y la reconstrucción obtenida con la configuración reducida de Gabor.

La Figura 7.2 muestra el espectrograma asociado a la configuración reducida. En este caso, la reconstrucción conserva parte de la energía principal del audio, pero todavía se observan diferencias importantes frente al espectrograma original.

En contraste, la Figura 7.3 presenta la forma de onda reconstruida con la configuración final. La señal reconstruida se ajusta mucho mejor a la señal original, lo cual coincide con el incremento observado en SNR y SI-SDR.

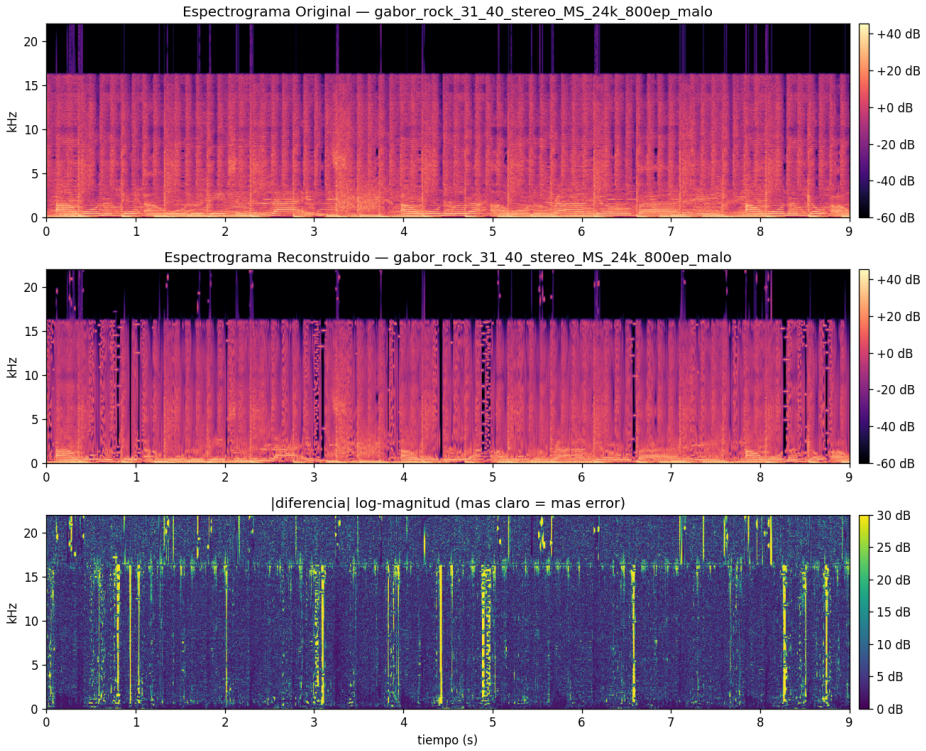


FIGURA 7.2: Espectrograma del experimento con configuración reducida de Gabor.

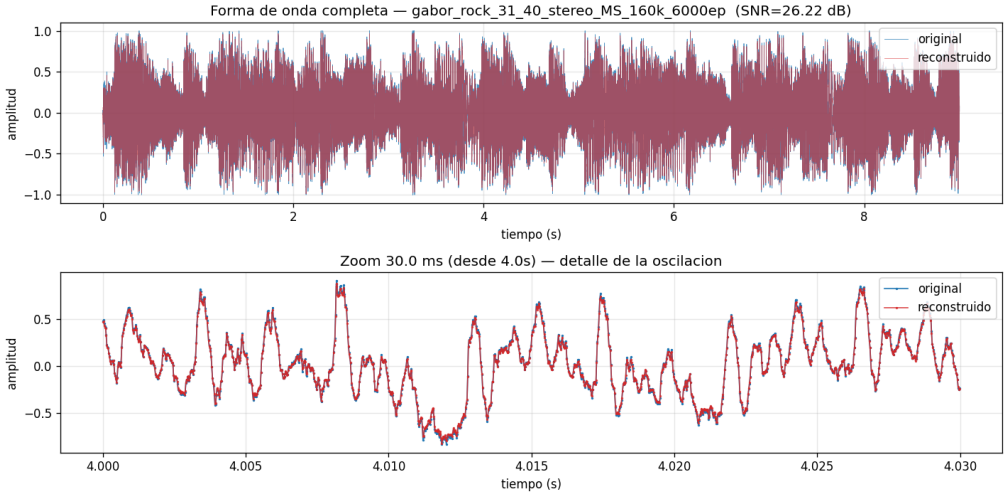


FIGURA 7.3: Comparación de la forma de onda original y la reconstrucción obtenida con la configuración final de Gabor.

La Figura 7.4 muestra el espectrograma de la configuración final. En comparación con la configuración reducida, la estructura espectral se conserva mejor y el error visual es menor.

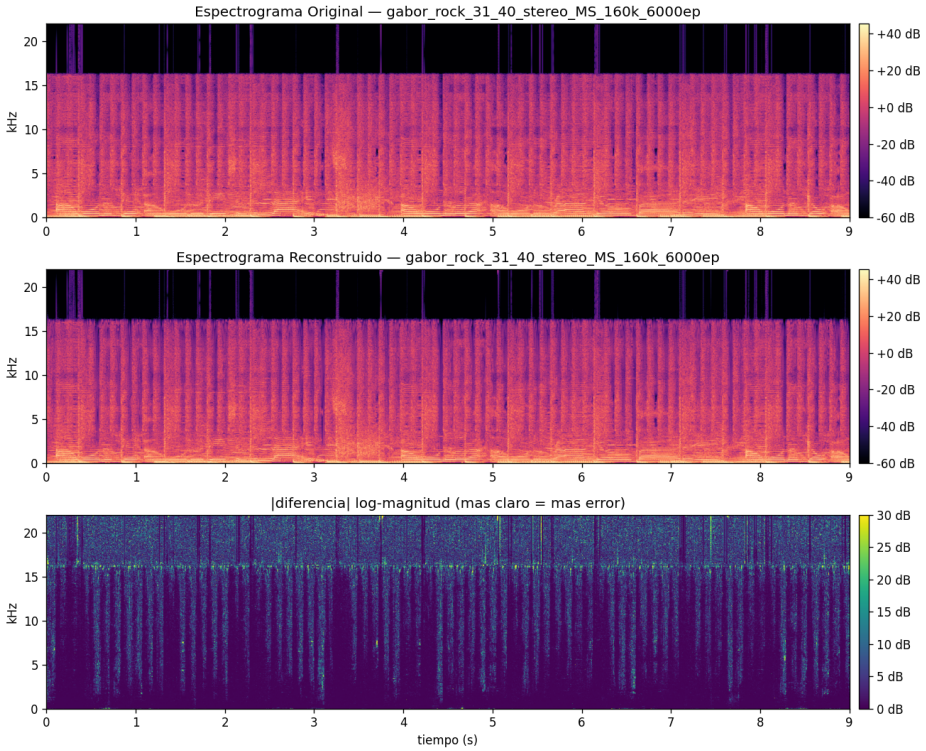


FIGURA 7.4: Espectrograma del experimento con configuración final de Gabor.

Video demostrativo del experimento de audio con Gabor: la comparación entre la configuración reducida y la configuración final se encuentra disponible en: https://www.youtube.com/watch?v=P3lGuTy_jHY

En conjunto, estos resultados muestran por qué se eligió el modelo de átomos de Gabor para la parte de audio. El intento inicial con gaussianas espectrales y Chebyshev permitió reutilizar parte de la idea del video, pero tenía problemas con la fase, dependía de Griffin-Lim y no alcanzaba una calidad suficiente. En cambio, Gabor permitió reconstruir directamente la forma de onda, incorporar la fase dentro del propio modelo y obtener mejores resultados en menor tiempo.

Finalmente, esta parte del trabajo deja abierta una línea interesante para investigación futura. El audio no fue el eje principal de esta tesis, por lo que todavía se podrían explorar más configuraciones, estrategias de compresión, etc. Aun así, los resultados obtenidos muestran que las primitivas explícitas también pueden ser útiles para representar señales de audio.

CAPÍTULO VIII

Representación multimedia integrada (audio + video)

8.1 Integración del pipeline: Chebyshev y átomos de Gabor

A partir de los experimentos realizados de forma independiente para video y audio, se consolidó un *pipeline* audiovisual que integra ambas ramas del sistema. En la rama visual, el video se representa mediante gaussianas 2D temporales cuyos atributos evolucionan en el tiempo usando polinomios de Chebyshev. En la rama de audio, la señal sonora se representa mediante átomos de Gabor, los cuales permiten aproximar oscilaciones localizadas en el tiempo.

El objetivo de esta integración fue probar el flujo completo del sistema usando como entrada un video MP4 y generando como salida una reconstrucción audiovisual. De esta manera, el sistema no solo reconstruye los *frames* del video, sino también el audio correspondiente al mismo intervalo temporal.

El *pipeline* recibe como entrada un archivo de video y tres archivos de configuración. El primero corresponde a la configuración general del *pipeline*, donde se define el segmento del video que será utilizado. El segundo corresponde al modelo de video y el tercero al modelo de audio. Con estos archivos se controla desde qué segundo hasta qué segundo se recorta el video, la resolución de salida, la cantidad de *frames*, la calidad de extracción y los parámetros de entrenamiento de cada rama.

El flujo general del sistema se puede resumir de la siguiente manera. Primero, se extrae un segmento del video original y se generan los *frames* en la resolución definida. En paralelo, se extrae el audio correspondiente al mismo intervalo temporal. Luego, el modelo de video se entrena usando gaussianas 2D temporales, mientras que

el audio se entrena usando átomos de Gabor. Finalmente, ambas reconstrucciones se combinan para generar un video final con audio.

En la rama de video, cada *frame* se reconstruye a partir de un conjunto de gaussianas 2D cuyos atributos cambian en el tiempo. La configuración final utiliza la base de Chebyshev, la pérdida con movimiento y el rasterizador acelerado en CUDA. En la rama de audio, la señal se reconstruye como una suma de átomos de Gabor optimizados por gradiente. Cada átomo representa una oscilación localizada, lo que permite aproximar directamente la forma de onda del audio.

Además del entrenamiento, el *pipeline* incluye una etapa de reducción del modelo de video. Después de entrenar la rama visual, se prueba la eliminación progresiva de gaussianas mediante *pruning*. Esta poda solo se conserva si la pérdida de calidad es pequeña según las métricas calculadas. Si la reducción afecta de forma considerable la calidad de reconstrucción, el sistema mantiene el modelo original o una versión menos agresiva.

Después del *pruning*, se aplica una etapa de cuantización de los parámetros del video. En particular, se genera una versión en formato `uint16`, lo que permite reducir el tamaño de almacenamiento de los parámetros respecto a una representación en punto flotante. Esta etapa acerca el modelo a una representación más compacta, aunque la compresión final todavía depende de la configuración usada y de la complejidad del video.

En conjunto, la integración permite trabajar con una misma idea general para dos tipos de señales distintas: representar contenido multimedia mediante primitivas explícitas y optimizables. En video, las primitivas son gaussianas 2D temporales; en audio, son átomos de Gabor.

8.2 Validación del sistema audiovisual

Es importante notar que las métricas del Capítulo VII y del Capítulo VIII no son directamente comparables, ya que fueron obtenidas sobre clips distintos.

El Capítulo VII evalúa audio aislado sobre un fragmento musical, mientras que este capítulo valida el funcionamiento del sistema audiovisual integrado sobre otro segmento.

Para evaluar el funcionamiento del sistema integrado, se realizó una prueba sobre un segmento audiovisual de 15 segundos a resolución 720p. En la rama de video se utilizaron 155000 gaussianas iniciales, 480 *frames* y 2000 épocas de entrenamiento. Los grados temporales empleados fueron 110/90/35/12/6/4 para posición, opacidad, color, escala, rotación y profundidad, respectivamente. La función de pérdida utilizada fue *motion*, con $\lambda_{\text{DSSIM}} = 0.25$ y $\lambda_{\text{motion}} = 2.0$.

Los resultados de video se muestran en la Tabla 8.1. El modelo obtuvo un PSNR promedio de 41.00 dB y un SSIM promedio de 0.977. Estos valores indican una reconstrucción visual de alta calidad. Además, el PSNR temporal promedio fue de 44.46 dB, lo que sugiere que el movimiento reconstruido mantiene una buena coherencia temporal respecto al video original.

Tabla 8.1: Métricas de reconstrucción de video en el sistema audiovisual.

Métrica	Valor
Gaussianas finales	155000
PSNR promedio	41.00 dB
PSNR mínimo	28.50 dB
PSNR p5	32.69 dB
SSIM promedio	0.977
LPIPS promedio	0.044
PSNR temporal promedio	44.46 dB

En la rama de audio se utilizó una representación estéreo en dominio *Mid-Side* (MS). Esta representación separa la señal en una componente central y una componente lateral, lo que permite distribuir los átomos de forma más eficiente. Para este experimento se utilizaron 160000 átomos en total, divididos en 96000 átomos para la componente *Mid* y 64000 átomos para la componente *Side*. El audio

fue procesado a una frecuencia de muestreo de 44100 Hz y entrenado durante 6000 épocas.

Los resultados de audio se presentan en la Tabla 8.2. El modelo obtuvo un SNR estéreo de 22.25 dB y un PSNR estéreo de 38.14 dB. Estos resultados muestran que la reconstrucción conserva una parte importante de la señal original, aunque el audio sigue siendo una rama más sensible debido a sus oscilaciones rápidas y a la percepción auditiva.

Tabla 8.2: Métricas de reconstrucción de audio en el sistema audiovisual.

Métrica	Valor
Dominio	MS
Átomos totales	160000
Átomos Mid	96000
Átomos Side	64000
Frecuencia de muestreo	44100 Hz
SNR estéreo	22.25 dB
SI-SDR estéreo	22.23 dB
PSNR estéreo	38.14 dB
LSD promedio	7.46 dB

La Figura 8.1 muestra una comparación entre la señal de audio original y la señal reconstruida. Esta comparación permite observar si la forma general de la onda fue conservada por el modelo.

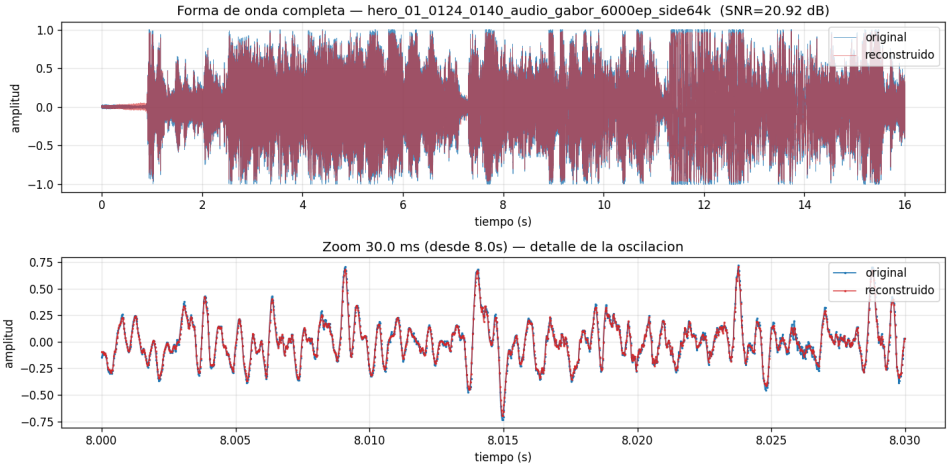


FIGURA 8.1: Comparación entre la señal de audio original y la señal reconstruida mediante átomos de Gabor.

La Figura 8.2 muestra la comparación entre los espectrogramas del audio original y reconstruido. Esta visualización es útil porque permite analizar si el modelo conserva la distribución de frecuencias a lo largo del tiempo.

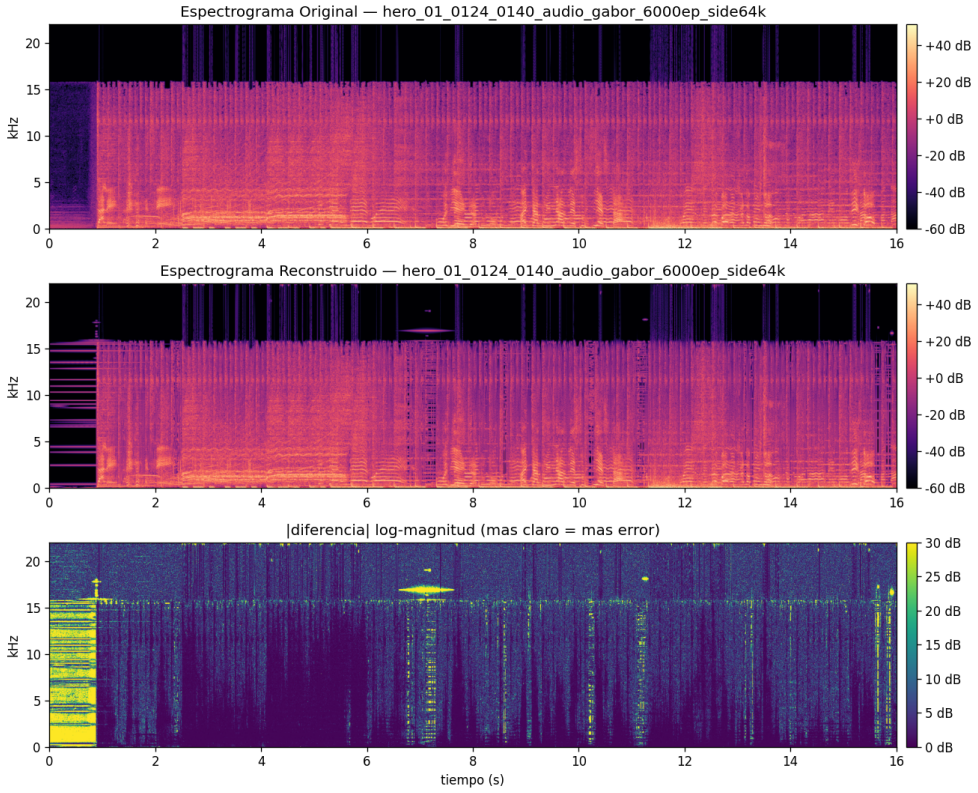


FIGURA 8.2: Comparación entre el espectrograma del audio original y el espectrograma del audio reconstruido.

Además de evaluar las métricas de video y audio por separado, se generó una salida audiovisual final. Esta salida combina los *frames* reconstruidos por el modelo de gaussianas temporales con la señal de audio reconstruida por el modelo de Gabor. De esta manera, se valida que ambas ramas pueden integrarse en un mismo resultado multimedia.

En el caso del video, también se generaron visualizaciones de las gaussianas más importantes. Para ello, se compararon dos *frames* del video original con la reconstrucción completa y con reconstrucciones parciales usando subconjuntos de 80k y 20k gaussianas. Esta comparación permite observar qué parte de la estructura visual se mantiene al reducir la cantidad de gaussianas utilizadas.

Las Figuras 8.3 y 8.4 muestran esta comparación visual en dos instantes distintos del video.



FIGURA 8.3: Comparación visual del *frame* 23 entre el video original, la reconstrucción completa y las reconstrucciones usando subconjuntos de 80k y 20k gaussianas importantes.



FIGURA 8.4: Comparación visual del *frame* 162 entre el video original, la reconstrucción completa y las reconstrucciones usando subconjuntos de 80k y 20k gaussianas importantes.

En ambos casos se observa que la reconstrucción completa conserva mejor los detalles del video original. Sin embargo, las reconstrucciones con 80k y 20k gaussianas todavía mantienen parte importante de la escena. Esto sugiere que no todas las gaussianas aportan de la misma manera a la reconstrucción, por lo que existe margen para seguir explorando estrategias de reducción del modelo.

En conjunto, esta prueba muestra que el sistema puede integrar ambas ramas en una salida audiovisual. La calidad final depende directamente de la configuración utilizada para video y audio: una mayor cantidad de gaussianas o átomos, más épocas y una selección adecuada de grados temporales pueden mejorar la reconstrucción, pero también aumentan el costo computacional.

Por tanto, el resultado consolidado no debe interpretarse únicamente como una prueba de compresión, sino como una demostración de que la representación por primitivas optimizables puede extenderse de imagen y video hacia una señal audiovisual completa. El video se representa mediante gaussianas 2D temporales y el

audio mediante átomos de Gabor, manteniendo una misma idea general: reconstruir señales multimedia a partir de componentes explícitos y optimizables.

Video demostrativo del framework audiovisual completo: el resultado final, integrando la reconstrucción de video mediante gaussianas 2D temporales y la reconstrucción de audio mediante átomos de Gabor, se encuentra disponible en: https://www.youtube.com/watch?v=iZ74_VLYOvA

CAPÍTULO IX

Conclusiones y trabajo futuro

9.1 Conclusiones

En esta tesis se presentó MBB-GS, una propuesta para representar video mediante gaussianas 2D temporales. La idea principal fue dejar de tratar el video solo como una lista de *frames* independientes y, en su lugar, describirlo como una señal visual continua. Para lograrlo, se mantuvo una misma población de gaussianas durante toda la secuencia, haciendo que sus atributos cambien en el tiempo mediante polinomios de Chebyshev.

El resultado más importante del trabajo es que esta idea sí es viable. El modelo logró reconstruir secuencias de video usando gaussianas 2D cuyos parámetros dependen del tiempo. Además, al tener una representación continua, fue posible evaluar el modelo no solo en los *frames* originales, sino también en instantes intermedios. Esto permitió explorar aplicaciones como la interpolación temporal y la generación de cámara lenta sin entrenar un interpolador externo.

Uno de los primeros aprendizajes fue que la elección de la base temporal importa bastante. La base monomial permitió representar cambios en el tiempo, pero presentó más dificultades de optimización y una reconstrucción menos estable. En cambio, la base de Chebyshev produjo mejores resultados cuantitativos y visuales. Por esta razón, Chebyshev terminó siendo la base principal del modelo temporal.

También se comprobó que la capacidad del modelo depende tanto del número de gaussianas como de los grados temporales. Al aumentar la cantidad de gaussianas, el modelo puede cubrir mejor los detalles espaciales del video. Al aumentar los grados, puede describir cambios temporales más complejos. Sin embargo, esta mejora no es gratis: más gaussianas y más coeficientes implican más memoria, más parámetros y mayor tiempo de entrenamiento. Por eso, una parte importante

del trabajo consistió en encontrar configuraciones que mejoren la reconstrucción sin hacer que el costo sea demasiado alto.

Otro resultado relevante fue el uso de una pérdida con peso por movimiento. En los videos evaluados, las zonas dinámicas suelen ser las más difíciles de reconstruir, ya que concentran cambios de posición, apariencia y oclusiones. Al dar más importancia a esas regiones durante el entrenamiento, el modelo pudo mejorar la reconstrucción de partes con movimiento. Este resultado fue importante porque mostró que no basta con reducir el error promedio del frame completo; también es necesario guiar al modelo hacia las regiones donde el video cambia más.

La interpolación temporal fue una de las aplicaciones más interesantes de la representación. Como los atributos de las gaussianas son funciones del tiempo, el modelo puede evaluarse en timestamps intermedios. Esto permitió generar versiones con más *frames* a partir del mismo checkpoint, sin entrenar una red adicional. Aun así, esta aplicación todavía tiene limitaciones. En escenas con movimiento rápido, pueden aparecer rastros, zonas borrosas o artefactos, lo que indica que reconstruir bien los *frames* originales no siempre garantiza que la trayectoria intermedia sea perfecta.

También se exploró la reducción del modelo mediante poda y cuantización. Esta parte mostró que es posible eliminar gaussianas con menor aporte visual y reducir el tamaño de la representación. En una prueba, el modelo pasó aproximadamente de 236 MB a 85 MB manteniendo una salida muy cercana a la reconstrucción completa. Este resultado muestra que existe potencial para reducir el modelo, aunque todavía no debe interpretarse como un sistema completo de compresión de video. Más bien, debe verse como una primera señal de que la representación explícita permite estudiar qué primitivas son más importantes y cuáles pueden retirarse o almacenarse de forma más eficiente.

Como extensión complementaria, se trabajó una rama de audio. Al inicio se intentó adaptar directamente la idea de video usando gaussianas espectrales temporales con Chebyshev sobre la magnitud del espectrograma. Ese intento fue útil

porque mostró rápidamente una limitación importante: el modelo solo aprendía magnitud y no fase. Por ello, para volver a una señal temporal era necesario usar la fase original o estimar la fase con métodos externos. Además, la calidad obtenida no fue suficiente y el entrenamiento resultaba lento para el nivel de reconstrucción alcanzado.

A partir de esa experiencia, se cambió el enfoque hacia átomos de Gabor. Esta representación fue más adecuada para audio porque trabaja directamente sobre la forma de onda e incorpora amplitud, duración, frecuencia y fase en cada átomo. En las pruebas realizadas, Gabor logró resultados claramente mejores que el intento espectral inicial y permitió reconstruir audio en menos tiempo. Por eso, dentro del alcance de esta tesis, los átomos de Gabor fueron la alternativa elegida para la rama de audio.

De todas formas, es importante remarcar que el audio no fue el eje principal del trabajo. La rama de audio se desarrolló como una extensión exploratoria para probar si la idea de primitivas optimizables también podía funcionar fuera del video. Los resultados fueron prometedores, pero todavía queda mucho por explorar antes de considerar esta parte como una solución completa para representación de audio.

Finalmente, se integraron las ramas de video y audio en una salida multimedia. El video fue representado mediante gaussianas 2D temporales y el audio mediante átomos de Gabor. Esta integración mostró que ambas modalidades pueden reconstruirse usando primitivas explícitas optimizables. En conjunto, MBB-GS muestra que este tipo de representación puede servir como una alternativa para modelar señales multimedia, con el video como aporte principal y el audio como una línea complementaria con potencial.

9.2 Limitaciones y trabajo futuro

Aunque los resultados son prometedores, el trabajo todavía tiene varias limitaciones. La primera está relacionada con la interpolación temporal. El modelo

puede evaluarse en instantes intermedios, pero eso no significa que todos los movimientos se interpolen correctamente. En movimientos rápidos o con cambios bruscos de apariencia, la reconstrucción intermedia puede presentar rastros, deformaciones o zonas borrosas. Esto muestra que la continuidad temporal aprendida por el modelo todavía puede mejorar.

Una dirección futura importante es incorporar restricciones más fuertes sobre la evolución temporal de las gaussianas. Por ejemplo, podrían explorarse pérdidas que penalicen trayectorias inestables, regularizaciones sobre movimiento o criterios que ayuden a mantener la coherencia entre *frames*. También sería útil evaluar la interpolación con videos más difíciles, con oclusiones, cámaras en movimiento y objetos que cambian rápidamente de forma o iluminación.

Otra limitación es el costo computacional. En general, aumentar el número de gaussianas, los grados temporales o las épocas de entrenamiento mejora la calidad, pero también incrementa el tiempo de entrenamiento y el uso de memoria. Por eso, una línea futura sería estudiar formas automáticas de asignar capacidad al modelo. No todos los videos necesitan la misma cantidad de gaussianas ni los mismos grados para todos los atributos. Un sistema más adaptativo podría usar más capacidad en zonas complejas y menos en regiones simples o casi estáticas.

La reducción del modelo también queda como una línea abierta. En esta tesis se probaron poda y cuantización, pero todavía falta desarrollar un esquema completo de compresión. Para acercarse a un códec real, sería necesario estudiar cómo codificar los parámetros, cómo ordenar o agrupar gaussianas, cómo aprovechar redundancias temporales y cómo comparar de forma justa contra formatos de video tradicionales. Por ahora, los resultados deben entenderse como una exploración inicial de reducción del modelo, no como una solución final de compresión.

También sería importante ampliar la evaluación experimental. Aunque se probaron distintas configuraciones, los experimentos se realizaron sobre un conjunto limitado de clips. En trabajos futuros convendría evaluar escenas más largas, videos con más texturas, cambios de iluminación, movimientos de cámara, fondos complejos

y distintos tipos de contenido. Esto permitiría entender mejor en qué casos MBB-GS funciona bien y en qué casos necesita ajustes.

En la rama de audio, la principal limitación es que el estudio todavía fue inicial. Los átomos de Gabor funcionaron mejor que el intento basado en espectrogramas, pero eso no significa que el problema de audio esté resuelto. La señal sonora tiene detalles perceptuales finos, transientes y componentes de alta frecuencia que pueden requerir más capacidad o estrategias más especializadas. Además, todavía falta evaluar el modelo con más canciones, voces, sonidos ambientales y fragmentos de mayor duración.

Como trabajo futuro en audio, se podrían explorar inicializaciones más robustas, modelos multibanda, nuevas pérdidas perceptuales y mejores formas de distribuir átomos entre las componentes Mid y Side en audio estéreo. También sería interesante comparar esta representación con métodos modernos de síntesis, compresión o reconstrucción de audio, para entender mejor qué ventajas reales ofrece y en qué escenarios puede ser competitiva.

Otra posibilidad es mejorar la integración audiovisual. En esta tesis, video y audio se entrenaron como ramas separadas y luego se combinaron en una salida final. En el futuro, se podría estudiar una representación más integrada, donde ambas modalidades compartan información temporal o se optimicen bajo criterios comunes de sincronización. Esto podría ser útil para clips donde los eventos visuales y sonoros están fuertemente relacionados.

Finalmente, también queda trabajo por hacer en reproducibilidad y organización del framework. Una versión futura podría incluir scripts más automatizados para lanzar experimentos, guardar configuraciones, registrar métricas, generar comparaciones y reconstruir resultados. Esto facilitaría repetir pruebas, comparar configuraciones y extender el sistema a nuevos videos o audios.

En resumen, MBB-GS muestra que las gaussianas 2D temporales pueden representar video como una señal continua y que esta idea puede extenderse, de

forma exploratoria, hacia audio y contenido multimedia. El trabajo deja resultados prometedores, pero también deja abiertas varias preguntas importantes sobre interpolación, eficiencia, reducción del modelo, calidad perceptual y evaluación en escenarios más variados.

REFERENCIAS BIBLIOGRÁFICAS

[1] B. Kerbl, G. Kopanas, T. Leimkühler, and G. Drettakis, "3d gaussian splatting for real-time radiance field rendering," *ACM Transactions on Graphics*, vol. 42, no. 4, pp. 1–14, 2023.

[2] X. Zhang, X. Ge, T. Xu, D. He, Y. Wang, H. Qin, G. Lu, J. Geng, and J. Zhang, "GaussianImage: 1000 fps image representation and compression by 2d gaussian splatting," in *European Conference on Computer Vision*, 2024.

[3] D. Gabor, "Theory of communication," *Journal of the Institution of Electrical Engineers*, vol. 93, no. 26, pp. 429–457, 1946.

[4] S. G. Mallat and Z. Zhang, "Matching pursuits with time-frequency dictionaries," *IEEE Transactions on Signal Processing*, vol. 41, no. 12, pp. 3397–3415, 1993.

[5] L. Wang, Y. Shi, and W. T. Ooi, "GSVC: Efficient video representation and compression through 2d gaussian splatting," in *Proceedings of the 35th Workshop on Network and Operating System Support for Digital Audio and Video*, 2025.

[6] I. Lee, Y. Choi, and J. Lee, "GaussianVideo: Efficient video representation and compression by gaussian splatting," in *IEEE/CVF Conference on Computer Vision and Pattern Recognition Workshops*, 2025, pp. 4471–4480.

[7] M. Liu, Q. Yang, M. Zhao, H. Huang, L. Yang, Z. Li, and Y. Xu, "D2GV: Deformable 2d gaussian splatting for video representation in 400fps," arXiv preprint arXiv:2503.05600, 2025.

[8] Z. Pang, Z. Ke, X. Zhou, and T. Qiu, "GSVR: 2d gaussian-based video representation for 800+ fps with hybrid deformation field," arXiv preprint arXiv:2507.05594, 2025.

[9] H. Jiang, D. Sun, V. Jampani, M.-H. Yang, E. Learned-Miller, and J. Kautz, "Super slomo: High quality estimation of multiple intermediate frames for video interpolation," in *IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2018, pp. 9000–9008.

[10] Z. Huang, T. Zhang, W. Heng, B. Shi, and S. Zhou, "RIFE: Real-time intermediate flow estimation for video frame interpolation," in *European Conference on Computer Vision*, 2022.

[11] M. Shi, X. Di, L. Peng, B. Cao, A. Wu, Z. Feng, J. Guo, R. Pei, X. Fu, Y. Cao, and Z. Zha, "GS-STVSR: Ultra-efficient continuous spatio-temporal video super-resolution via 2d gaussian splatting," arXiv preprint arXiv:2604.18047, 2026.

[12] D. Griffin and J. Lim, "Signal estimation from modified short-time fourier transform," *IEEE Transactions on Acoustics, Speech, and Signal Processing*, vol. 32, no. 2, pp. 236–243, 1984.

[13] J. Engel, L. Hantrakul, C. Gu, and A. Roberts, "DDSP: Differentiable digital signal processing," in *International Conference on Learning Representations*, 2020.

[14] R. Yamamoto, E. Song, and J.-M. Kim, "Parallel WaveGAN: A fast waveform generation model based on generative adversarial networks with multi-resolution spectrogram," in *IEEE International Conference on Acoustics, Speech and Signal Processing*, 2020, pp. 6199–6203.

[15] J. Le Roux, S. Wisdom, H. Erdogan, and J. R. Hershey, "SDR – half-baked or well done?" in *IEEE International Conference on Acoustics, Speech and Signal Processing*, 2019, pp. 626–630.

Apéndice A

Repositorio de código y reproducibilidad

1.1 Disponibilidad del código fuente

El código fuente desarrollado como parte de esta investigación se encuentra disponible públicamente en el repositorio oficial de MBB-GS:

github.com/JLeandroJM/MBB-GS

El repositorio reúne las implementaciones utilizadas para desarrollar y evaluar la representación multimedia propuesta en esta tesis. Su publicación tiene como objetivo facilitar la revisión técnica del trabajo, la reproducción de los experimentos y la extensión futura del framework.

1.2 Contenido general del repositorio

El repositorio contiene los principales componentes computacionales utilizados durante la investigación, entre ellos:

- la implementación del modelo de gaussianas bidimensionales con atributos dependientes del tiempo;
- la evaluación de los atributos temporales mediante polinomios de Chebyshev;
- el rasterizador diferenciable desarrollado y acelerado mediante CUDA;
- los módulos necesarios para el entrenamiento y la reconstrucción de secuencias de video;
- la implementación de la representación de audio mediante átomos de Gabor;

- los componentes CUDA utilizados para acelerar el cálculo de la señal de audio y de sus gradientes;
- los scripts de entrenamiento, evaluación, reconstrucción e interpolación temporal;
- los archivos de configuración empleados en los distintos experimentos presentados en esta tesis.

La descripción detallada de la formulación temporal y del funcionamiento del rasterizador se presenta en el Capítulo IV. Los experimentos relacionados con la representación de video, sus aplicaciones y la representación de audio se desarrollan en los capítulos correspondientes del documento.

1.3 Versión del código asociada con la tesis

Debido a que el repositorio puede continuar recibiendo modificaciones después de la presentación de esta tesis, se recomienda utilizar una versión identificable del código para reproducir los resultados reportados.

La versión asociada con la entrega final del documento corresponde a:

- **Repositorio:** `github.com/JLeandroJM/MBB-GS`.
- **Rama principal:** `main`.

El identificador del commit permite acceder al estado exacto del código utilizado en los experimentos, incluso si posteriormente se realizan cambios en la rama principal del proyecto.

1.4 Requisitos generales de ejecución

La implementación requiere un entorno compatible con Python, PyTorch y CUDA. Debido a que el framework incluye extensiones desarrolladas específicamente para GPU, es necesario utilizar una tarjeta gráfica NVIDIA y una instalación de CUDA compatible con la versión de PyTorch empleada.

De manera general, el proceso de ejecución comprende las siguientes etapas:

1. clonar el repositorio del proyecto;
2. crear el entorno de ejecución e instalar las dependencias;
3. compilar las extensiones CUDA del rasterizador o del modelo de audio;
4. preparar los frames de video o la señal de audio de entrada;
5. seleccionar el archivo de configuración correspondiente;
6. ejecutar el entrenamiento;
7. reconstruir la señal y calcular las métricas de evaluación.

Las instrucciones actualizadas de instalación, compilación y ejecución se encuentran en el archivo `README.md` del repositorio.

1.5 Datos, resultados y material complementario

Los archivos de configuración y los scripts necesarios para ejecutar los experimentos se encuentran disponibles en el repositorio. Sin embargo, algunos videos originales, resultados completos, checkpoints y archivos generados durante el entrenamiento pueden ocupar una cantidad considerable de almacenamiento.

Por esta razón, el repositorio prioriza la publicación del código fuente, las configuraciones y las instrucciones necesarias para reproducir el procedimiento.